

Lecture Notes on Deep Neural Networks

Jawwad Ahmed Shamsi
FAST–NUCES, Pakistan

February 16, 2026

1 Fundamentals of Machine Learning

In the first week, we will study fundamentals of Machine Learning. The chapter has following learning objectives:

1. Define AI/ML/DL/DS
2. Explain supervised vs unsupervised learning
3. Understand K-means steps
4. Feature Engineering
5. Compute Manhattan/Euclidean distance
6. Compute accuracy, precision, recall, F1 and interpret threshold tradeoff

1.1 Difference between AI vs. ML vs. DL

AI is a concept to create intelligent machines that can simulate human thinking capability and behavior. It provides the ability to mimic human cognitive thinking such as problem-solving and capability of a computer system to mimic human cognitive behavior and make decisions.

ML is a subset of AI, which provides us statistical tools to explore the data and gives ability to learn. It is a field of study that gives computers the ability to learn through mathematical models without being explicitly programmed.

DL is a subset of ML, which alleviates the need for feature engineering. It utilizes the concept of neural networks to learn and stimulates human brain.

1.2 Fundamental Concepts of Machine Learning

We would like to have a program, which can learn from experience.

The output of Machine learning would be a program.

Machine Learning is a field of study that gives computers the ability to learn without being explicitly programmed (Arthur Samuel, 1959). Figure 2 illustrates the difference between a traditional program and a machine learning program.

1.3 Applications of Machine Learning

There are several examples of Applications of Machine Learning. These include:

1. Natural Language Processing
2. Computer Vision
3. Computational Biology
4. Recommender systems by Netflix and Amazon
5. Google ads.

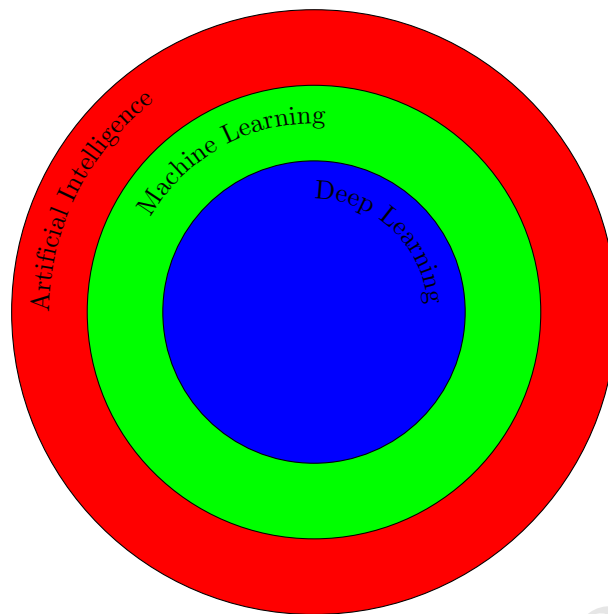


Figure 1: Relationship between AI, Machine Learning, and Deep Learning.

6. Drug Discovery
7. Character recognition by Post Office
8. Assisted Driving

A machine learning model could either have a linear relationship between input and output or it could have been a non-linear relationship.

1.4 Linear vs Non-linear

Linear relationship is modeled through a straight line.

Non-Linear relationship can be represented through any non-linear function. For instance, following are examples of non-linear functions:

1. Quadratic functions: $f(x) = ax^2 + bx + c$
2. Exponential functions: $f(x) = a \cdot e^x$
3. Logarithmic functions: $f(x) = \log(x)$
4. Trigonometric functions: $f(x) = \sin(x)$, $f(x) = \cos(x)$

Most of the practical examples in our life are based on non-linear functions. Some of the examples are as follows:

1. Traffic Flow: Speed of a vehicle on road does not change linearly with the number of other cars on the road
2. Weight loss: Weight loss or weight gain does not change linearly with exercise
3. Passing a course: Your grades in a course do not change linearly with the number of hours studied for the course.

1.5 Types of Machine Learning Problems

Machine Learning Problems can be classified as Supervised learning or Unsupervised learning.

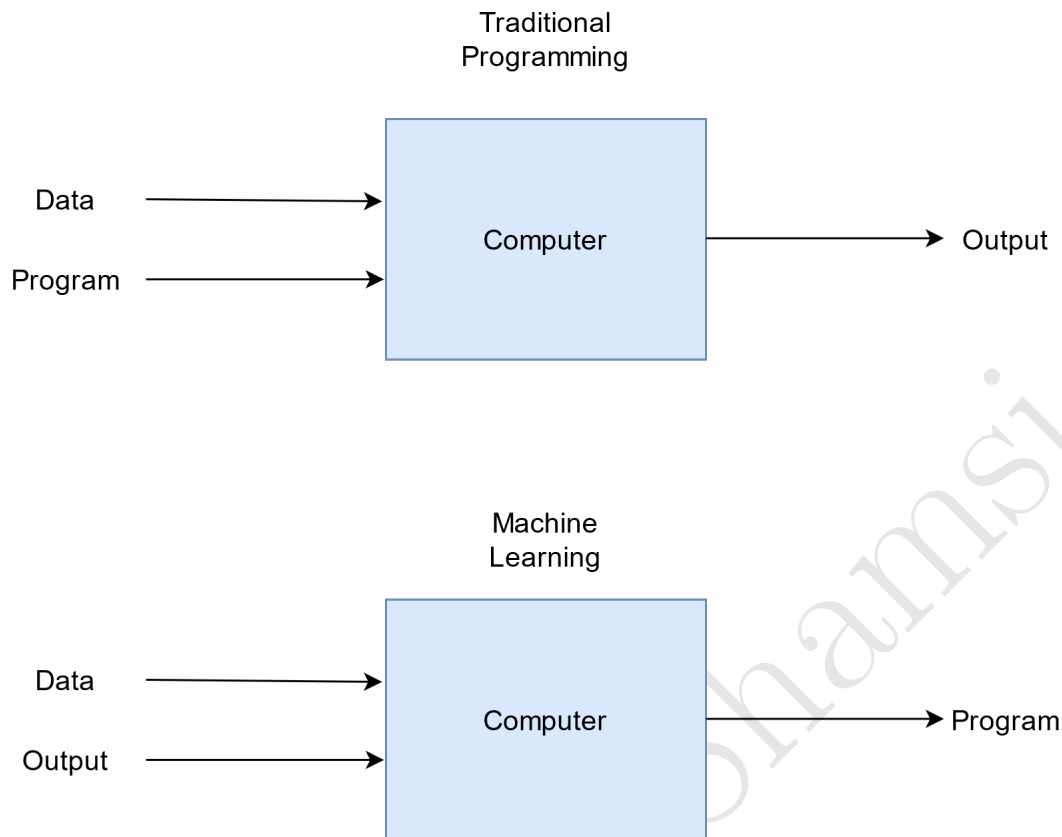


Figure 2: ML vs Prog

1.5.1 Supervised learning

In these problems, data is labeled. In such problems, given a set of features, the task is to find the rule which will predict the label for unseen data.

Types of Supervised Learning

1. Classification : Predict the class
2. Regression: Predict the value

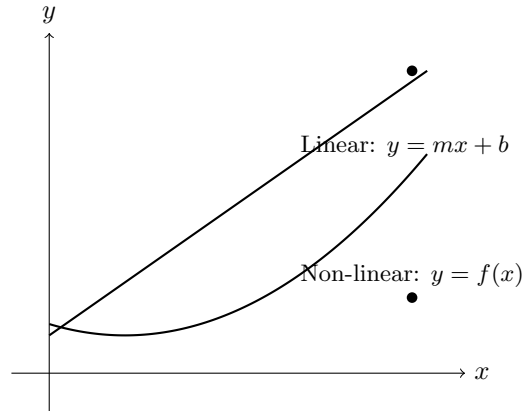
Example of Classification : Predict grades based on number of hours studied **Example of Regression:** Predict price of land according to area.

1.5.2 Clustering or Unsupervised learning

In this class of machine learning algorithms, the data is not labeled. Let us group them based on features and see the pattern.

K-Means = A popular clustering algorithm Given a set of unlabeled feature vectors, group them into groups.

1. Suppose there are K clusters
2. Pick random k points as center points or centroids
3. Assign each data points to centroids
4. cluster remaining samples by minimizing distance between the data points in the same cluster
5. Recompute each centroid as the mean of the assigned points.

Figure 3: Linear vs Non-linear relationship between input x and output y .

Suppose we have a few data points and we arranged them in to two clusters. Later, we have two new data points, one of them fits well into one of the clusters, whereas the other one is at the edge of the circle. In such a case, we have two options. First, either to re-cluster all the data points or to have more than two clusters.

Let's understand the K-means algorithm through an example: **Step 1:** Input data

Table 1: Data for K-Means

Point	Value
P1	(2,3)
P2	(3,3)
P3	(6,8)
P4	(7,8)
P5	(2,2)
P6	(3,2)
P7	(6,7)
P8	(7,7)
P9	(3,4)
P10	(6,6)

Step 2: Randomly initialize two centroids for $k=2$. Let's choose:

- **Centroid C1:** (2, 2)
- **Centroid C2:** (6, 8)

Step 3: Compute the distance and assign the points to the nearest centroid. For instance, if Euclidean distance is used then the distance from each point to the centroids can be calculated as follows (first few points):

$$d(x, y) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Table 2: Point Distances and Assigned Clusters

Point	Distance to C1 (2,2)	Distance to C2 (6,8)	Assigned Cluster
P1	$\sqrt{(2-2)^2 + (3-2)^2} = 1.0$	$\sqrt{(6-2)^2 + (8-3)^2} = 6.4$	C1
P2	$\sqrt{(3-2)^2 + (3-2)^2} = 1.4$	$\sqrt{(6-3)^2 + (8-3)^2} = 5.8$	C1
P3	$\sqrt{(6-2)^2 + (8-2)^2} = 6.48$	$\sqrt{(6-8)^2 + (8-8)^2} = 2.0$	C2
P4	$\sqrt{(7-2)^2 + (8-2)^2} = 7.8$	$\sqrt{(7-6)^2 + (8-8)^2} = 1.0$	C2

step 4: Compute centroid of each cluster by computing mean of all the points in the cluster

step 5: Iterate step 3. Stop when the algorithm is converged.

Final cluster assignment

- **Cluster 1:** P1, P2, P5, P6, P9
- **Cluster 2:** P3, P4, P7, P8, P10

1.6 Data

Some questions:

1. What is the training data and how are we going to evaluate it?
2. what is the training data and how are we going to evaluate the success of the system?
3. How do we figure what are the right features and how do we measure distances between them. How to find similarities?

Data can be split into training, testing, and validation:

1. **Training data** This refers to the portion of data which is used to train a model. A model train on one set of data (may or) may not perform with similar accuracy on another set of data.
2. **Validation data:** The portion of data which is used to assess a model's accuracy or performance during the process of training is called as validation data. The primary purpose is to validate the process of training through unseen data. Training is considered comprehensive (and complete) if the training accuracy is acceptable and is close to the validation accuracy.
3. **Testing Data** This refers to the portion of data which is used to assess a model's quality or it's efficiency. Never use test data during model design or hyperparameter tuning (to avoid data leakage).

1.7 Features

Features are attributes about data point. Selection of right features are very important in Machine learning.

"All models are wrong but some are useful". George Box

Suppose I would like to predict grades in this course. Features such as prior programming experience, existing CGPA, no. of lectures attended are important. some other features such as color of eyes or residential address may seem irrelevant and may lead to overfitting.

We can input all the features and sorts them to find out which are useful. However, it may lead to overfitting. So goal is to maximize the features which are likely to be useful and minimize those which are not useful.

Let's take an example: We have been given five features and we have to develop a classifier which can identify if a given animal is a reptile or not.

Table 3: Feature Selection Example

Name	Egg-Laying	Scales	Poisonous	Cold-blooded	no. of legs	reptile
Cobra	True	True	True	True	0	Yes
Rattlesnake	True	True	True	True	0	Yes
Boa Constrictor	False	True	False	True	0	Yes
Chicken	True	True	False	False	2	No
Alligator	True	True	False	True	4	Yes
Dart Frog	True	False	True	False	4	No
Salmon	True	True	False	True	0	No
Python	True	True	False	True	0	Yes

Consider table For the first row, we start by selecting all the five features. These are highlighted in red. This continued for row 2. In row 3, we have a different case. Boa Constrictor does not lay eggs, it is not poisonous and it is a reptile. So we refined the selected features to three. In row 5, we refined the selected features by changing the requirement of no. of legs to either 0 or 4. The problem arises in row 7 and row 8. They both have similar

features but the response variable is different. We consider this as an anomaly or outlier. In the final selection, we have reduced the no. of features to two, i.e. scales and Cold-blooded.

Design Choice: No False negatives. That is, no instance, which is a reptile and we call it not a reptile. We may have false positives, i.e., an instance that is not a reptile and we call it as a reptile (example:salmon).

In general, feature selection affects both accuracy and generalization.

1.8 Computing Distance

Distance will be used to group items or how to separate them a part.

Another way to separate reptiles and non-reptiles is to measure the distance between different data points. This is specifically useful if the data is unlabeled.

A simple way is to convert them into feature vectors. Goal is to either classify them through a divider or to cluster them accordingly. Given the data points above, following are the feature vectors of each of the above animals:

1. Cobra [1,1,1,1,0]
2. rattlesnake[1,1,1,1,0]
3. boa constrictor[0,1,0,1,0]
4. dart frog[1,0,1,0,4]

The question is how to measure distance between these feature vectors? There are two popular methods. That is, Manhattan Distance and Euclidean Distance

1.8.1 Manhattan

Manhattan: Take the sum of the absolute values of the differences of the coordinates.

As per NIST, Manhattan distance is defined as

The distance between two points measured along axes at right angles

For two points $P(x_1, y_1)$ and $Q(x_2, y_2)$, the Manhattan distance is:

Manhattan Distance = $|x_2 - x_1| + |y_2 - y_1|$

Manhattan distance is preferred over Euclidean distance when data is of high dimension.

Example:: Consider two points:

- P(2,3)
- Q(6,8)

The Manhattan distance is calculated as:

$$|6 - 2| + |8 - 3| = 9$$

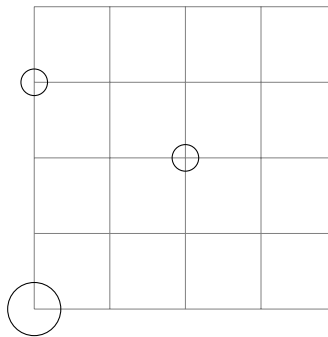


Figure 4: Manhattan Distance.

1.8.2 Euclidean Distance

For Euclidean distance, we simply compute the square root of the sum of the squares of the differences of the coordinates.

For instance, if $x=(a,b)$ and $y=(c,d)$, the Euclidean distance between x and y is

$$d(x, y) = \sqrt{(a - c)^2 + (b - d)^2} \quad (1)$$

In general, for n -dimensional data, Euclidean distance can be expressed as follows:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (2)$$

Feature Engineering: choice of features matters too many features may lead to overfitting weights may have an impact

comparing distances using the following feature vector Alligator [1,1,0,1,4] rattlesnake [1,1,1,1,0] boa Constrictor [0,1,0,1,0] dart frog [1,0,1,0,4]

Table 4: Distance Computation

	rattlesnake	boa constrictor	dart frog	alligator
rattlesnake	-	1.414	4.243	4.123
boa Constrictor	1.414	-	4.472	4.123
dart frog	4.243	4.472	-	1.732
alligator	4.123	4.123	1.732	-

In table 4 distance between Alligator and rattlesnake and alligator and boa constrictor is quite high. Although all the three animals are reptiles.

Using binary features in table 5, i.e. an animal has either got legs or no legs, we may get a more realistic value

Table 5: Distance Calculation using binary

	rattlesnake	boa constrictor	dart frog	alligator
rattlesnake	-	1.414	1.732	1.414
boa constrictor	1.414	-	2.236	1.414
dart frog	1.732	2.236	-	1.732
alligator	1.414	1.414	1.732	-

1.8.3 Evaluation Metrics for Classification

In classification problems, the predictions of a model can be summarized using a **confusion matrix**. For binary classification, we define:

- **True Positive (TP):** Model predicts Positive and the sample is actually Positive.
- **False Positive (FP):** Model predicts Positive but the sample is actually Negative.
- **True Negative (TN):** Model predicts Negative and the sample is actually Negative.
- **False Negative (FN):** Model predicts Negative but the sample is actually Positive.

For binary classification, the confusion matrix summarizes prediction results as follows:

Actual \ Predicted	Positive (1)	Negative (0)
Positive (1)	True Positive (TP)	False Negative (FN)
Negative (0)	False Positive (FP)	True Negative (TN)

Interpretation:

- **TP:** Correctly predicted Positive
- **TN:** Correctly predicted Negative
- **FP:** Incorrectly predicted Positive (Type-I error)
- **FN:** Incorrectly predicted Negative (Type-II error)

Common Evaluation Metrics:1. **Accuracy:**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

2. **Precision:**

$$\text{Precision} = \frac{TP}{TP + FP} \quad (4)$$

Precision answers: “Out of all predicted positives, how many were actually positive?”

3. **Recall (Sensitivity):**

$$\text{Recall} = \frac{TP}{TP + FN} \quad (5)$$

Recall answers: “Out of all actual positives, how many did we correctly identify?”

4. **F1-Score:**

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (6)$$

1.8.4 Worked Example: Accuracy, Precision, Recall, and F1

Assume we are building a spam detector where:

- **Positive class** = Spam
- **Negative class** = Not Spam

Suppose we tested our model on 100 emails and obtained the following results:

$$TP = 9, \quad FP = 1, \quad TN = 79, \quad FN = 11$$

Note that the total number of samples is:

$$TP + FP + TN + FN = 9 + 1 + 79 + 11 = 100$$

1) **Accuracy**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{9 + 79}{100} = \frac{88}{100} = 0.88$$

2) **Precision**

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{9}{9 + 1} = \frac{9}{10} = 0.90$$

3) **Recall**

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{9}{9 + 11} = \frac{9}{20} = 0.45$$

4) **F1-score**

$$F1 = 2 \cdot \frac{(0.90)(0.45)}{0.90 + 0.45} = 2 \cdot \frac{0.405}{1.35} = 2 \cdot 0.30 = 0.60$$

1.8.5 Threshold and Precision–Recall Tradeoff

Many classifiers output a probability score $p(y = 1|x)$, where $y = 1$ represents the positive class. We choose a **decision threshold** t :

- Predict Positive if $p(y = 1|x) \geq t$
- Predict Negative if $p(y = 1|x) < t$

Why Precision and Recall trade off:

- If we increase the threshold (make the model strict), fewer samples are predicted as Positive. This usually reduces FP (precision increases), but increases FN (recall decreases).
- If we decrease the threshold (make the model relaxed), more samples are predicted as Positive. This usually reduces FN (recall increases), but increases FP (precision decreases).

1.8.6 Spam Filter Example: Strict vs Relaxed Threshold

Assume we have 100 emails:

- Actual spam emails = 20
- Actual not-spam emails = 80

Case A: Strict threshold ($t = 0.9$) The model predicts only 10 emails as spam:

$$TP = 9, FP = 1, FN = 11, TN = 79$$

$$\text{Precision} = \frac{9}{9+1} = 0.90, \quad \text{Recall} = \frac{9}{9+11} = 0.45$$

This gives **high precision** but **low recall**. In practice, this means: “When the filter says spam, it is usually correct, but many spam emails still appear in the inbox.”

Case B: Relaxed threshold ($t = 0.3$) Now the model predicts 30 emails as spam:

$$TP = 18, FP = 12, FN = 2, TN = 68$$

$$\text{Precision} = \frac{18}{18+12} = 0.60, \quad \text{Recall} = \frac{18}{18+2} = 0.90$$

This gives **high recall** but **lower precision**. In practice, this means: “Most spam is caught, but some normal emails may be incorrectly moved to the spam folder.”

Conclusion:

- **Strict threshold** $\Rightarrow$ high precision, low recall
- **Relaxed threshold** $\Rightarrow$ high recall, lower precision

1.9 Class Imbalance

A class imbalance problem occurs if we have imbalance of occurrence of classes. For instance, in case of a cancer detection dataset we have fewer instances of cancer. Class imbalance occurs when the distribution between the two classes is highly skewed. Normally, we have more negative instances as compared to the positive instances.

1.9.1 Choosing the right set of parameters

Accuracy can be misleading on imbalanced data. For instance, consider if we have a highly skewed data such as only one positive per hundred negatives. If we develop a model, which always predicts a negative value, such a model will give us 99% accuracy. However, the model will not be acceptable. Accuracy is not reliable when classes are imbalanced.

The failure of accuracy as a metric on imbalanced data is well-known. Consider the case of a dataset with the ratio of 1 positive per 100 negatives. On this task, a model that predicts all cases to be negative yields an accuracy of 99%. This model, nevertheless, is a dummy classifier that always predicts the majority class.

1.10 Summary

We have studied basics of Machine Learning. These include initial concepts, feature Engineering, K-Means, and Distance Calculation

Exercises

1. **AI vs ML vs DL:** Give **one real-life example** of each:

- (a) Artificial Intelligence (AI)
- (b) Machine Learning (ML)
- (c) Deep Learning (DL)

2. **Linear vs Non-linear:** For each function below, state whether it is **linear** or **non-linear**.

- (a) $y = 3x + 2$
- (b) $y = x^2 + 1$
- (c) $y = \log(x)$
- (d) $y = 5$

3. **Supervised vs Unsupervised:** For each problem below, identify whether it is **supervised learning** or **unsupervised learning**.

- (a) Predicting house price using area and location
- (b) Grouping students into clusters based on study habits
- (c) Detecting spam emails
- (d) Finding customer segments from purchase history

4. **Manhattan Distance:** Compute the Manhattan distance between the following points:

$$P(1, 2), \quad Q(6, 5)$$

5. **Euclidean Distance:** Compute the Euclidean distance between the following points:

$$P(1, 2), \quad Q(6, 5)$$

6. **Confusion Matrix $\rightarrow$ Evaluation Metrics:** A binary classifier is tested on 50 samples and produces the following results:

$$TP = 12, \quad FP = 3, \quad TN = 30, \quad FN = 5$$

Compute:

- (a) Accuracy
- (b) Precision
- (c) Recall
- (d) F1-score

7. **Precision vs Recall (Conceptual):** In a **cancer detection system**, which metric is more important and why?

- (a) Precision
- (b) Recall

8. **Threshold Tradeoff:** A classifier outputs a probability score $p(y = 1|x)$ and predicts the positive class when:

$$p(y = 1|x) \geq t$$

If the threshold is increased from $t = 0.3$ to $t = 0.9$, explain what happens to:

- (a) False Positives (FP)
- (b) False Negatives (FN)
- (c) Precision
- (d) Recall

9. **Class Imbalance:** A dataset contains 1000 samples:

- 990 samples belong to the negative class
- 10 samples belong to the positive class

A model predicts **all samples as negative**.

- (a) What is the accuracy of the model?
- (b) Why is this model not useful in practice?
- (c) Which metric(s) would better reflect this failure?

2 Fundamentals of Deep Neural Networks

In this chapter, we will study basics of Deep Neural Network (DNN). Let's start with the basic building.

2.1 A Neuron

An Artificial Neural Network (ANN) consists of many neurons, which are structured in layers. Each neuron takes one or many inputs, performs some computation, and gives an output. The term Multi-layer Perceptron (MLP) is interchangeably used with ANN. A neuron performs the following functions:

1. Takes the input signals
2. Computes the weighted sum of the inputs to represent the total strength
3. Applies an activation function
4. Checks whether to trigger an output if the threshold exceeds a limit

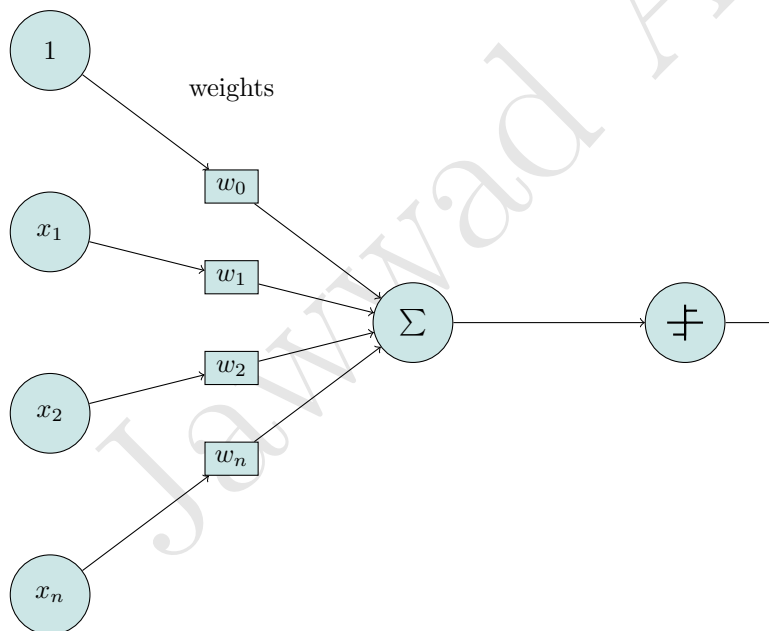
Weights are important. They provide significance of features. Not all features are equally important. During our training phase, the ANN will try to learn the correct value of the weights.

The weighted sum function is the sum of all inputs multiplied by their weights and then add a bias term.

$$z = \sum_{i=1}^n x_i w_i + b \quad (7)$$

$$z = x_1 w_1 + x_2 w_2 + x_3 w_3 + \cdots + x_n w_n + b \quad (8)$$

where bias is the y-intercept. It allows us to better fit our model Recall: $y=mx+b$ inputs



Activation Function

What is an activation function It is a function, which triggers the output of a neuron. It takes the input weighted sum and the bias and decides to give an output provided the input computation exceeds a certain threshold.

How does a perceptron learn:

A perceptron estimates a value. However, the value may not be correct. It performs an iterative cycle in which it computes the error (i.e., the difference between estimated value and the actual value). The weights and bias are adjusted based on the error. Below is the outline procedure: predicted value.

2.2 Training of Neuron

1. The neuron calculates the weighted sum and applies an activation function to make a prediction:

$$y_e = \text{activation} \left(\sum_{i=1}^n x_i w_i + b \right) \quad (9)$$

2. The prediction error is computed as:

$$\text{error} = y_e - y_p \quad (10)$$

where y_e is the estimated output and y_p is the ground-truth (actual) output.

3. The weights are updated. If the prediction is too high, weights are adjusted to make a lower prediction and vice versa.
4. Repeat until the error becomes minimal (or within an acceptable bound).

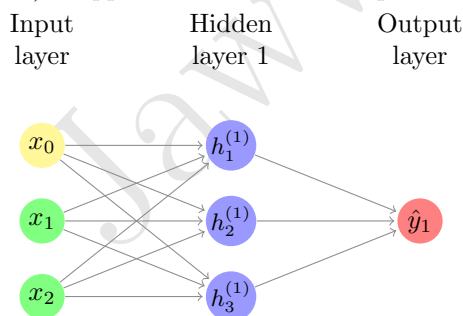
Error Functions are calculated differently, depending upon the type of output.

- For regression: Mean Square error
- For categorical values: Cross-entropy

We will revert to these calculations, later. During the training process, acceptable values of weights are determined. These weights are often referred to as parameters.

A neural network could consist of several neurons, organized in the form of hidden layers and output layer. A major distinction between a Neural Network and a Deep Neural Network (DNN) is the number of hidden layers. A DNN could consist of a large number of hidden layers.

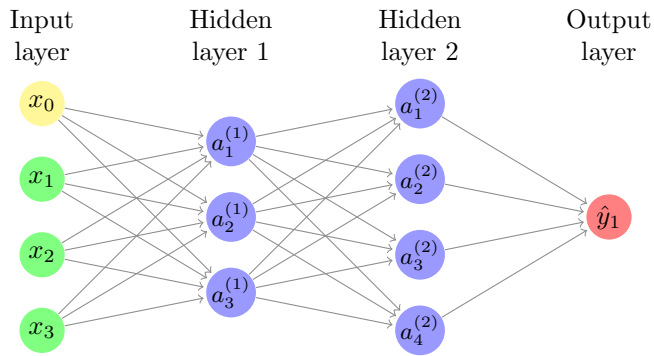
The example below shows a neural network with three neurons in the hidden layer. Each neuron computes a linear function of the input. If no activation function is applied, the output of each neuron corresponds to a straight line (or hyperplane). Combining multiple such linear neurons without a non-linear activation function still results in an overall linear model. Non-linear behavior emerges only when a non-linear activation function (such as ReLU or sigmoid) is applied to the neuron outputs.



MLP consists of the following:

1. Input vector (or features)
2. Hidden Layers
3. Weight Connections
4. Output Layer

A fully connected Neural Network is shown below:



Why do we need more than one neuron in a hidden layer?

A single neuron (even with a non-linear activation function) can only learn a limited pattern. Multiple neurons allow the network to learn multiple features in parallel. Each neuron has its own set of weights and bias, and therefore responds to different regions of the input space. Multiple neurons increase the expressiveness of the network by allowing it to model more complex and diverse patterns.

Why do we need more than one hidden layer?

Because different layers learn representations at different levels of abstraction. Earlier layers typically learn simple patterns, while deeper layers combine these patterns to learn more complex features.

Why do we need non-linear activation functions?

We need non-linearity because most of the problems, which we solve through deep learning are non-linear. Further, in a DNN with several hidden layers, linear activation function (in hidden layers) becomes useless. Without a non-linear activation function, stacking multiple layers would still result in a linear model.

2.3 Activation Functions

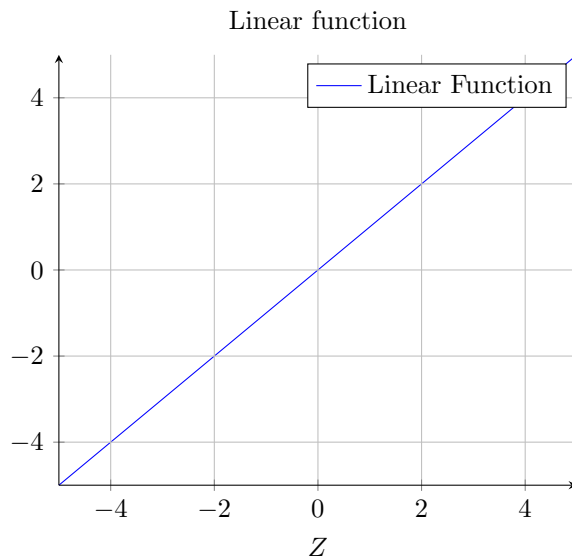
Now we will study different activation functions. Remember that the purpose of the activation function is to trigger an output when the weighted sum of inputs (and the bias) exceed certain threshold. This helps in achieving non-linearity.

2.3.1 Linear

Linear functions are not useful for DNN

example of linear function include

$$f(x) = x$$

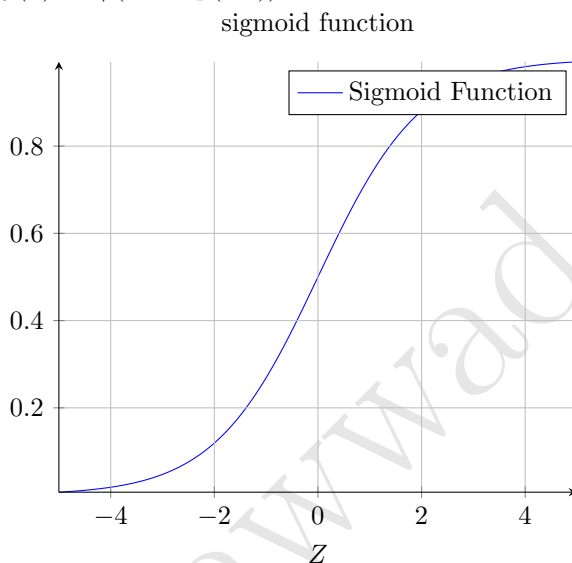


2.3.2 Sigmoid

It is used for binary classification.

It outputs probabilities. It is a differentiable function, i.e., slope can be found between any two points.

$$f(x) = 1/(1 + \exp(-x))$$



The sigmoid activation function outputs probabilities for the occurrence of an event. The threshold is set as 0.5. If the output probability is ≤ 0.5 then the event is not detected, whereas an output probability of > 0.5 implies that the event has been detected.

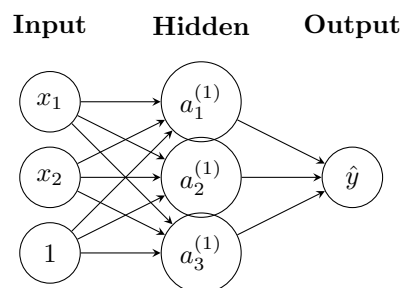


Figure 5: A simple neural network with one hidden layer.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

Table 6: Truth Table of OR gate

Example: Sigmoid Function (OR Gate)**Sigmoid Function**

Consider an OR gate of two inputs x_1 and x_2 . The truth table of OR gate is shown above. The equation for the above figure is shown below:

$$y = x_1w_1 + x_2w_2 + b \quad (11)$$

Devise weights w_1, w_2 and bias b .

Solution: For $x_1 = 0$ and $x_2 = 0$, the impact of w_1, w_2 is negated, and b should be negative because the input to the sigmoid function should be negative. This will yield an output less than 0.5.

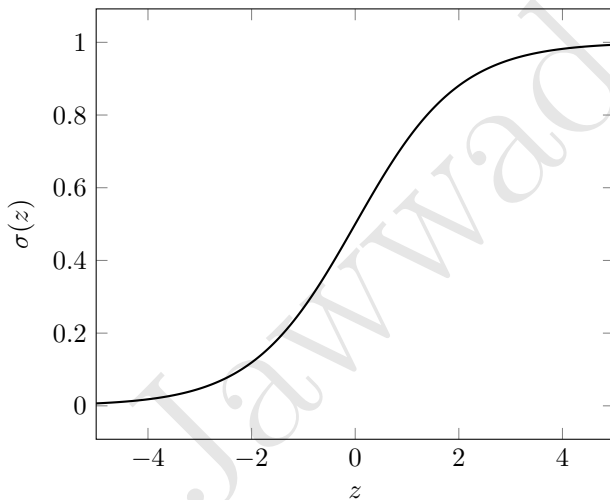
For all other values of x_1, x_2 , the combined effect of $w_1x_1 + w_2x_2$ should be greater than $-b$ (remember b is negative).

Some suggested values which satisfy both conditions are:

$$w_1 = 1, w_2 = 1, b = -0.5$$

or

$$w_1 = 2, w_2 = 2, b = -1$$

2.3.3 Softmax Activation Function

The **softmax** activation function is used for **multi-class classification**. It converts a vector of real-valued scores (called *logits*) into probabilities such that:

- Each probability lies in the range $[0, 1]$
- The sum of all probabilities is equal to 1

The softmax function is defined as:

$$\text{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Example: Grade Prediction Using Softmax

Consider the dataset shown in Table 7.

Student ID	Hours Studied	Attendance (%)	Grade
1	5	90	A
2	3	75	B
3	1	60	C
4	4	85	B
5	2	65	C
6	6	95	A

Table 7: Student performance dataset

We want to predict the grade $\{A, B, C\}$ for a **new student** with:

- **Hours Studied** = 4
- **Attendance** = 80%

Neural Network Architecture

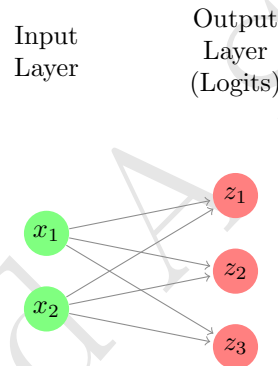


Figure 6: Single-layer neural network (softmax classifier) for grade prediction

Note: This network has no hidden layer and corresponds to a single-layer softmax classifier. Hidden layers will be introduced later. The output layer produces three values:

$$z_A, z_B, z_C$$

These values are called **logits** (raw scores before softmax).

Logit Computation

For each class $i \in \{A, B, C\}$, the logit is computed as:

$$Z_i = w_{i1} \cdot \text{Hours} + w_{i2} \cdot \text{Attendance} + b_i$$

Logit Calculation for the New Student

Grade A

$$Z_A = 0.5 \cdot 4 + 0.3 \cdot 80 + 1.0 = 2.0 + 24.0 + 1.0 = 27.0$$

Grade B

$$Z_B = 0.4 \cdot 4 + 0.4 \cdot 80 + 0.5 = 1.6 + 32.0 + 0.5 = 34.1$$

Grade C

$$Z_C = 0.3 \cdot 4 + 0.2 \cdot 80 - 0.5 = 1.2 + 16.0 - 0.5 = 16.7$$

Thus, the logit vector is:

$$Z = [27.0, 34.1, 16.7]$$

Softmax Computation**Step 1: Exponentiate Logits**

$$e^{Z_A} = e^{27.0} \approx 5.32 \times 10^{11}$$

$$e^{Z_B} = e^{34.1} \approx 6.48 \times 10^{14}$$

$$e^{Z_C} = e^{16.7} \approx 1.81 \times 10^7$$

Step 2: Sum of Exponentials

$$\sum e^{Z_i} \approx 6.49 \times 10^{14}$$

Step 3: Compute Probabilities

$$P(A) = \frac{e^{Z_A}}{\sum e^{Z_i}} \approx 0.00082 \text{ (0.082\%)}$$

$$P(B) = \frac{e^{Z_B}}{\sum e^{Z_i}} \approx 0.99985 \text{ (99.985\%)}$$

$$P(C) = \frac{e^{Z_C}}{\sum e^{Z_i}} \approx 2.79 \times 10^{-8}$$

Final Prediction

The predicted class is the one with the highest probability:

$$\text{Predicted Grade} = \arg \max\{P(A), P(B), P(C)\}$$

Predicted Grade = B

Key Insight Softmax does not introduce new weights. It only converts the logits into probabilities while preserving their relative order.

2.3.4 ReLU Activation Function

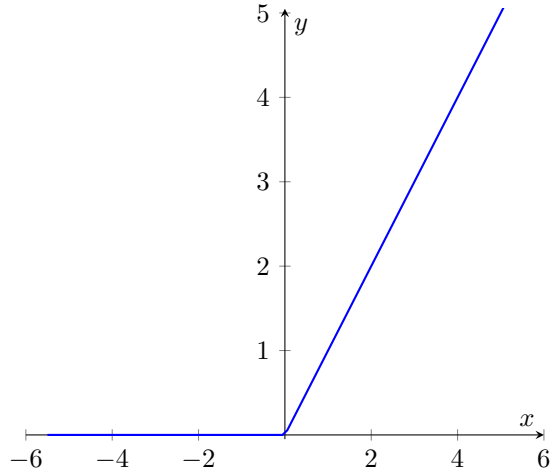
The Rectified Linear Unit (ReLU) is one of the most commonly used activation functions in deep learning. It is defined as:

$$\text{ReLU}(z) = \max(0, z) \quad (12)$$

This implies that:

- if the input z is negative, the output is zero,
- if the input z is positive, the output is equal to z .

ReLU introduces non-linearity into the neural network while remaining computationally simple.



Why ReLU is Important

ReLU has several advantages over traditional activation functions such as sigmoid and tanh:

- **Efficient computation:** ReLU requires only a simple thresholding operation, making it fast to evaluate.
- **Reduced vanishing gradient problem:** For positive inputs, the gradient of ReLU is constant, allowing gradients to propagate effectively through deep networks.
- **Sparse activation:** Since ReLU outputs zero for negative inputs, only a subset of neurons are active at any time, leading to efficient feature representations.

Geometric Interpretation

Geometrically, ReLU preserves the linear behavior for positive inputs while suppressing negative responses. As a result, different neurons activate in different regions of the input space, allowing the network to learn piecewise linear decision boundaries.

Limitation of ReLU

A known limitation of ReLU is the **dying ReLU problem**. If a neuron consistently receives negative inputs, it may output zero for all inputs and stop learning because its gradient becomes zero.

To address this issue, several variants of ReLU have been proposed, including:

- Leaky ReLU
- Parametric ReLU (PReLU)

Summary

ReLU is the default activation function in many deep learning architectures due to its simplicity, computational efficiency, and effectiveness in training deep neural networks.

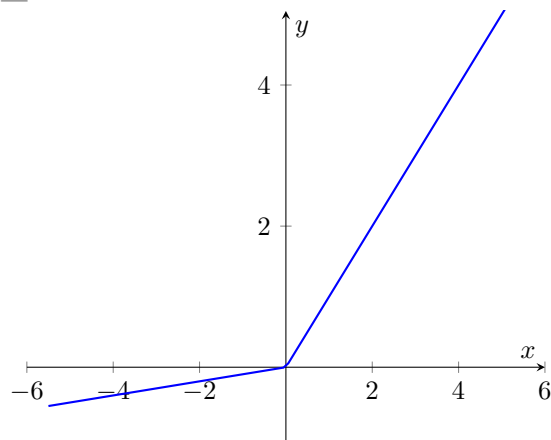
2.3.5 Leaky ReLU Activation Function

Leaky Rectified Linear Unit (Leaky ReLU) is a variant of the ReLU activation function designed to address the **dying ReLU problem**.

It is defined as:

$$\text{Leaky ReLU}(z) = \begin{cases} z, & \text{if } z > 0 \\ \alpha z, & \text{if } z \leq 0 \end{cases} \quad (13)$$

where α is a small positive constant (typically $\alpha = 0.01$).



Motivation

In standard ReLU, neurons output zero for all negative inputs, which can cause some neurons to stop learning if they consistently receive negative values. Leaky ReLU mitigates this issue by allowing a small, non-zero gradient when the input is negative.

Key Properties

- **Non-zero gradient for negative inputs:** Ensures that neurons continue to learn even when $z < 0$.
- **Computational simplicity:** Retains the efficiency of ReLU with minimal additional computation.
- **Improved robustness:** Reduces the risk of inactive (dead) neurons during training.

Comparison with ReLU

Unlike ReLU, which completely suppresses negative inputs, Leaky ReLU allows a small negative output. This results in smoother optimization and improved convergence in some deep networks.

Summary

Leaky ReLU improves upon ReLU by preserving a small gradient for negative inputs, helping prevent dead neurons and improving training stability in deep neural networks.

Is Leaky ReLU a Linear Function?

Leaky ReLU is **not** a linear function.

It is defined as:

$$\text{Leaky ReLU}(z) = \begin{cases} z, & \text{if } z > 0 \\ \alpha z, & \text{if } z \leq 0 \end{cases} \quad (14)$$

where $0 < \alpha < 1$.

Although each segment of the function is linear, the function as a whole is **piecewise linear** and therefore **non-linear**. This is because the slope of the function changes at $z = 0$.

If Leaky ReLU were a linear function, stacking multiple layers would still result in a linear model. The change in slope introduced by Leaky ReLU enables neural networks to model complex, non-linear decision boundaries.

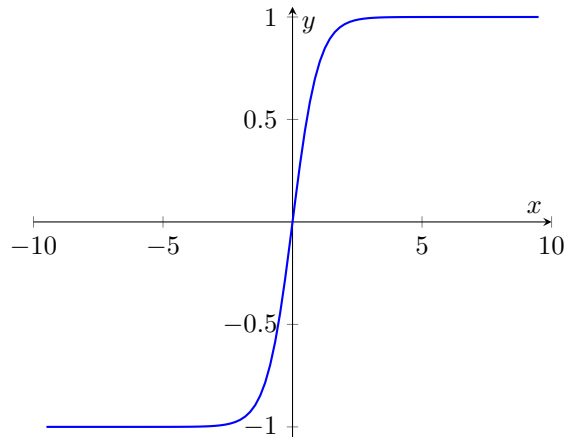
Key Point: Leaky ReLU is piecewise linear but not a linear function.

2.3.6 Tanh Activation Function

The hyperbolic tangent (tanh) activation function is a smooth, non-linear function commonly used in neural networks. It is defined as:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad (15)$$

The output of the tanh function lies in the range $(-1, 1)$.



Key Properties

- **Zero-centered output:** Unlike the sigmoid function, tanh produces outputs centered around zero, which helps reduce bias in weight updates.
- **Smooth non-linearity:** The function is differentiable everywhere, making it suitable for gradient-based optimization.
- **Bounded output:** Since the output is limited to $(-1, 1)$, tanh can help stabilize learning in some cases.

Geometric Interpretation

The tanh function behaves linearly near the origin and gradually saturates for large positive or negative values of z . This allows neurons to respond strongly to moderate inputs while suppressing extreme values.

Limitation of Tanh

A major limitation of the tanh activation function is the **vanishing gradient problem**. For large values of $|z|$, the gradient of tanh becomes very small, which slows down learning in deep networks.

Due to this limitation, tanh is less commonly used in deep feedforward networks but remains useful in certain architectures such as recurrent neural networks.

Summary

The tanh activation function introduces smooth non-linearity and produces zero-centered outputs, but its tendency to saturate can hinder learning in very deep neural networks.

Below is a summary extracted from the book

Activation functions

61

Table 2.1 A cheat sheet of the most common activation functions

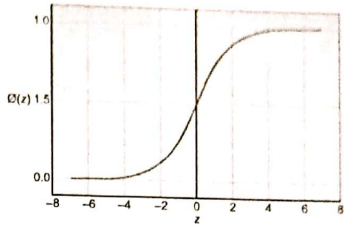
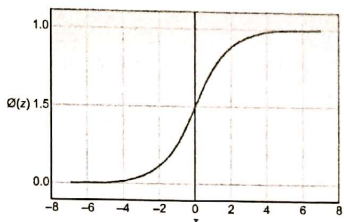
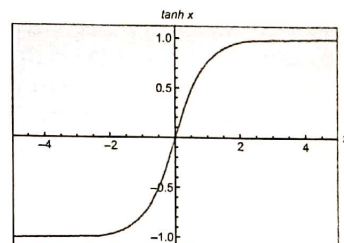
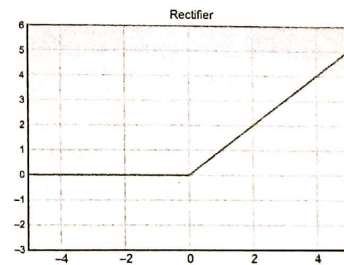
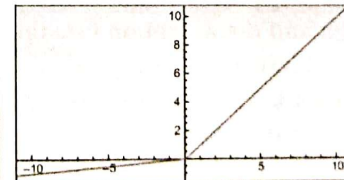
Activation function	Description	Plot	Equation
Sigmoid/ logistic function	Squishes all the values to a probability between 0 and 1, which reduces extreme values or outliers in the data. Usually used to classify two classes.		$\sigma(z) = \frac{1}{1 + e^{-z}}$
Softmax function	A generalization of the sigmoid function. Used to obtain classification probabilities when we have more than two classes.		$\sigma(x_j) = \frac{e^{x_j}}{\sum_i e^{x_i}}$
Hyperbolic tangent func- tion (tanh)	Squishes all values to the range of -1 to 1. Tanh almost always works better than the sigmoid function in hidden layers.		$\tanh(x) = \frac{\sinh(x)}{\cosh(x)}$ $= \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Rectified linear unit (ReLU)	Activates a node only if the input is above zero. Always recommended for hidden layers. Better than tanh.		$f(x) = \max(0, x)$
Leaky ReLU	Instead of having the function be zero when $x < 0$, leaky ReLU introduces a small negative slope (around 0.01) when (x) is negative.		$f(x) = \max(0.01x, x)$

Figure 7: Activation Functions

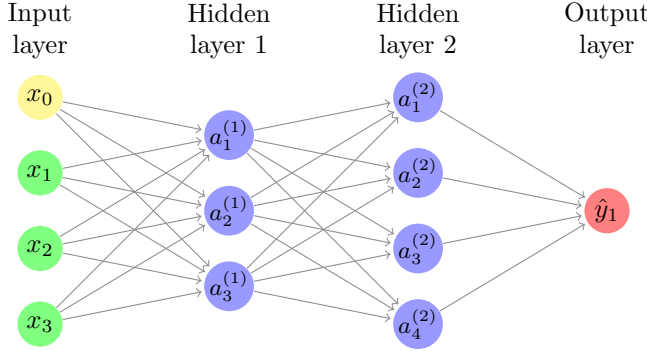
2.4 Multi-label vs multi-class

Having studied different types of activation functions, let's analyze key differences between multi-label and multi-class problems. Multi-class means a classification problem with more than two classes. These classes are distinct, i.e., only one of them could occur at once. For instance, in a corpus of images, we can distinguish between dog, cat, and fish. Since sum of all probabilities is 1 here, we should use the Softmax activation function.

In comparison, multi-label classification means labels which are not distinct and can occur together. For instance, in a Chest X-Ray scan system, multiple chest diseases such as T.B, Cancer, and CoVID can co-exist. In this scenario, sum of all probabilities can be greater than one. In this case, we will use the Sigmoid activation function.

2.5 Forward Propagation

Having studied activation functions, we will now focus on learning how propagation takes place in a neural network



$$a_1^1 = \sigma(w_{11}^1 \cdot x_1 + w_{12}^1 \cdot x_2 + w_{13}^1 \cdot x_3)$$

$$a_2^1 = \sigma(w_{21}^1 \cdot x_1 + w_{22}^1 \cdot x_2 + w_{23}^1 \cdot x_3)$$

$$a_3^1 = \sigma(w_{31}^1 \cdot x_1 + w_{32}^1 \cdot x_2 + w_{33}^1 \cdot x_3)$$

where w_{jk}^i denotes weight b/w the j^{th} node at the i^{th} layer and k^{th} node at the $i-1^{\text{th}}$ layer

$$\hat{y} = \sigma(w_{11}^3 a_1^2 + w_{12}^3 a_2^2 + w_{13}^3 a_3^2 + w_{14}^3 a_4^2)$$

We can also solve it using matrix.

$$= \sigma(z)$$

$$\begin{bmatrix} W_{11}^3 & W_{12}^3 & W_{13}^3 & W_{14}^3 \end{bmatrix} \cdot \sigma \left(\begin{bmatrix} W_{11}^2 & W_{12}^2 & W_{13}^2 \\ W_{21}^2 & W_{22}^2 & W_{23}^2 \\ W_{31}^2 & W_{32}^2 & W_{33}^2 \\ W_{41}^2 & W_{42}^2 & W_{43}^2 \end{bmatrix} \cdot \sigma \left(\begin{bmatrix} W_{11}^1 & W_{12}^1 & W_{13}^1 \\ W_{21}^1 & W_{22}^1 & W_{23}^1 \\ W_{31}^1 & W_{32}^1 & W_{33}^1 \end{bmatrix} \cdot \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix} \right) \right)$$

As we proceed in a deep neural network (DNN), multiple layers of weights are involved. Each layer performs two steps: a weighted sum followed by an activation function.

2.5.1 Layer-wise Computation

Hidden Layer 1 The pre-activation and activation of the first hidden layer are given by:

$$z_j^{(1)} = \sum_{i=1}^3 w_{ji}^{(1)} x_i \quad (16)$$

$$a_j^{(1)} = \sigma(z_j^{(1)}) \quad (17)$$

Hidden Layer 2

$$z_k^{(2)} = \sum_{j=1}^3 w_{kj}^{(2)} a_j^{(1)} \quad (18)$$

$$a_k^{(2)} = \sigma(z_k^{(2)}) \quad (19)$$

Output Layer

$$z^{(3)} = \sum_{k=1}^4 w_{1k}^{(3)} a_k^{(2)} \quad (20)$$

$$\hat{y} = \sigma(z^{(3)}) \quad (21)$$

Here, $w_{ij}^{(l)}$ represents the weight connecting the j -th neuron of layer $(l-1)$ to the i -th neuron of layer l .

2.5.2 Matrix Representation

The same computation can be written compactly as:

$$z^{(1)} = W^{(1)}x \quad (22)$$

$$a^{(1)} = \sigma(z^{(1)}) \quad (23)$$

$$z^{(2)} = W^{(2)}a^{(1)} \quad (24)$$

$$a^{(2)} = \sigma(z^{(2)}) \quad (25)$$

$$z^{(3)} = W^{(3)}a^{(2)} \quad (26)$$

$$\hat{y} = \sigma(z^{(3)}) \quad (27)$$

Note: Separating z (pre-activation) and a (activation) is essential for understanding backpropagation and gradient-based learning.

2.5.3 Why We Separate z and a

Separating pre-activation (z) and activation (a) is useful because:

- Backpropagation uses derivatives of activation functions
- Different activation functions can be swapped without changing the linear part
- It improves mathematical clarity in deep networks

Important Note: If a network has *no hidden layer*, then $a^{(1)}$ does not exist, and logits are computed directly from the input.

2.6 Error and Cost Function

We have studied that training is guided by the error, which is the difference between the predicted and actual values for a single data point. This error is transformed using a loss function. The cost function aggregates the loss over all training samples and provides a single measure of the model's overall performance. The example below

Cost function $J(w)$ is a function of weights. It is a non-convex function means that there could be more than one minima Figure 8 illustrates difference b/w a convex and a non-convex function. A non-convex function may

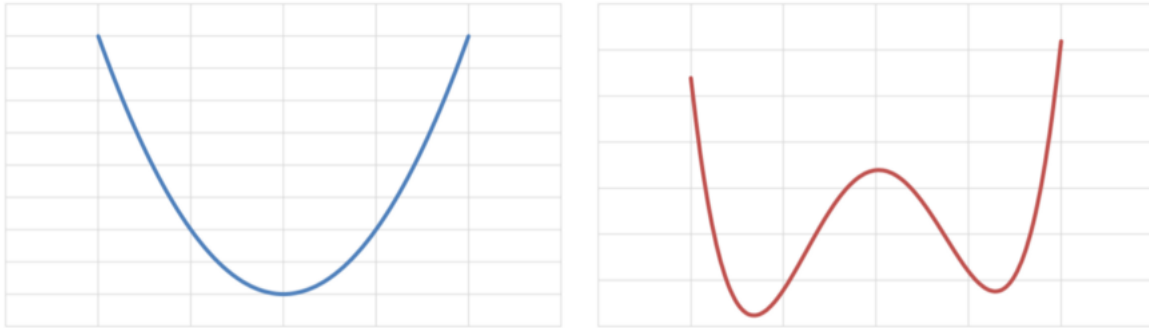


Figure 8: Convex vs Concave

have multiple global minima. Cost function in DNN is a non-convex function b/c of possibilities of multiple layers and different combination of weights Cost function is calculated as follows:

$$J(w) = |\hat{y} - y| \text{ i.e., absolute difference between the estimated } y \text{ and the calculated } y.$$

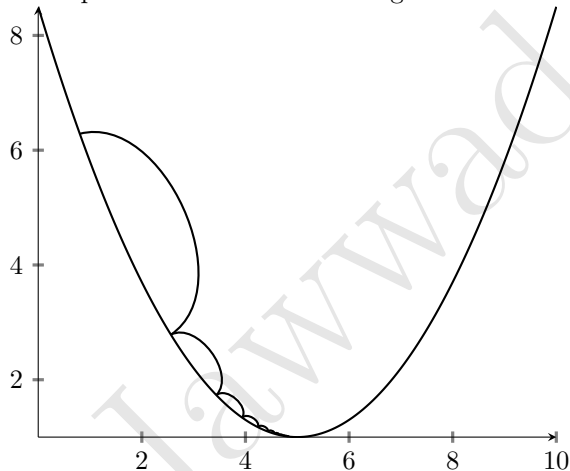
$$\hat{y} = w.x$$

assuming $x=0.4$ and $y=.7$

$$\hat{y} = w.0.4$$

$$J(w) = |w.0.4 - .7|$$

This equation is a function of weights



slope is important b/c it shows us the direction of adjustment of weights whereas η denotes the step size

Minima

Question Is it possible to have multiple minimums in a output of a DNN?

Yes. because a DNN could have multiple layers and at each layer we have different combinations of weights. So different combinations of weights could lead to the same output.

Error vs Loss vs Cost Function

The terms **error** and **cost function** are often used interchangeably in informal discussion, but they have different technical meanings in machine learning.

Error Error refers to the discrepancy between the predicted output and the true output for a *single training example*.

For example, for a single data point:

$$\text{Error} = \hat{y} - y \quad (28)$$

Error is defined at the level of an individual sample and does not describe the performance of the model on the entire dataset.

Loss Function A **loss function** maps the error of a single training example to a non-negative scalar value.

For example, the squared error loss is defined as:

$$\mathcal{L}(y, \hat{y}) = (y - \hat{y})^2 \quad (29)$$

The loss function quantifies how bad a prediction is for one sample.

Cost Function The **cost function** (also called the objective function) aggregates the loss over the entire training dataset.

For a dataset with N samples, the cost function is defined as:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y^{(i)}, \hat{y}^{(i)}) \quad (30)$$

The cost function provides a single measure of how well the model performs on the whole dataset and is the quantity minimized during training.

Summary

- **Error:** Difference between prediction and ground truth for one sample
- **Loss:** Function applied to the error of one sample
- **Cost:** Average (or sum) of the loss over all training samples

Key Point: Gradient descent minimizes the *cost function*, not the error of a single data point.

Now let's analyze different types of errors used and how error can be used to train a neural network. Recall that error is a difference between the actual value and the estimated value.

$$\text{Error} = |\hat{y} - y_i|$$

Error is always +ve. This is because we are interested in the absolute value.

Remember that the estimated value depends upon the weights and bias. So in other words, Error is a function of weights and bias.

2.6.1 Types of Errors

1. For Continuous Output Variable or Regression Problems:

- Mean Square Error $E(W, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y} - y_i)^2$
MSE penalizes outliers
- Or Absolute Mean Error $E(W, b) = \frac{1}{n} \sum_{i=1}^n |\hat{y} - y_i|$

2. For Categorical Values or Classification Problems

Cross Entropy is used to quantify the problem as it quantifies the difference between two probability distributions. We can use the following equation to compute the cross entropy loss.

$$E(W, b) = -\sum_{i=1}^m \hat{y}_i \log(p_i)$$

cross-entropy over all training examples can be computed as follows:

$$E(W, b) = -\sum_{i=1}^n \sum_{j=1}^m \hat{y}_{ij} \log(p_{ij})$$

Here $\hat{y}$ is the target probability (p) is the predicted probability and (m) is the number of classes.

Example**Understanding Cross-Entropy Loss**

Consider a classification problem where we want to distinguish between three classes: **Elephant**, **Lion**, and **Giraffe**.

Suppose a training instance belongs to the class **Lion**. The true (target) probability distribution is therefore:

- Class A (Elephant): 0.0
- Class B (Lion): 1.0
- Class C (Giraffe): 0.0

This is a one-hot encoded target vector, indicating 100% probability for the correct class.

Case 1: Poor Prediction

Assume the model predicts the following probability distribution:

- Class A (Elephant): 0.3
- Class B (Lion): 0.2
- Class C (Giraffe): 0.5

The categorical cross-entropy loss is computed as:

$$\mathcal{L} = - \sum_i y_i \log(\hat{y}_i) \quad (31)$$

Substituting the values:

$$\mathcal{L} = -(0.0 \cdot \log(0.3) + 1.0 \cdot \log(0.2) + 0.0 \cdot \log(0.5)) = -\log(0.2) \approx 1.609 \quad (32)$$

A high loss value indicates a poor prediction because the model assigns low probability to the correct class.

Case 2: Better Prediction

Now suppose the model predicts:

- Class A (Elephant): 0.2
- Class B (Lion): 0.6
- Class C (Giraffe): 0.2

The cross-entropy loss becomes:

$$\mathcal{L} = -\log(0.6) \approx 0.511 \quad (33)$$

The lower loss value indicates a better prediction.

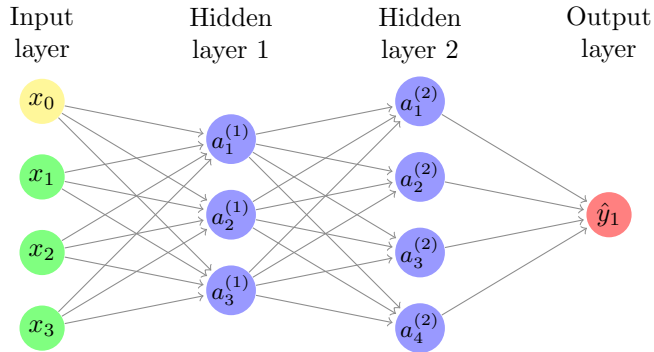
Observation: Cross-entropy loss decreases as the predicted probability of the correct class increases, and it heavily penalizes confident but incorrect predictions.

2.7 Adjusting Error

In order to reduce error, we have to adjust weights. There are two possibilities, a brute force approach or some algorithm to determine a possible set of weights.

2.7.1 Brute Force

Let's analyze brute force first.



In the above diagram, there are 28 possible weights and eight possible values of bias. This is a simple DNN. If we increase hidden layers or no. of neurons in the hidden layers then it will eventually increase the no. of neurons (and no of weights and bias). If we only consider 28 weights, and assume that each weight could have 1000 different values then we will have 10^{84} possible values of weights (1000X1000 X10001000, 25 times). This will increase with the increase in weights.

Certainly, the brute force approach is not scalable. This is because as the no. of neurons and no. of layers increase, no. of weights also increase as well.

So we need a mechanism to adjust weights

2.7.2 Local Minima Vs. Global Minima

Before diving deeper into the algorithm, let's recall the difference between the local minima and the global minima. The former is the value of the function (if plotted on a graph) which is less than all other values near it (local context). Whereas, the latter is a value of the function, which is less than all the other values of the function (global minima) The word minima is used as a plural and indicates the possibility of multiple values.

Example

Why do we have multiple local minima In a DNN, there are multiple combination of weights. So multiple minima are possible

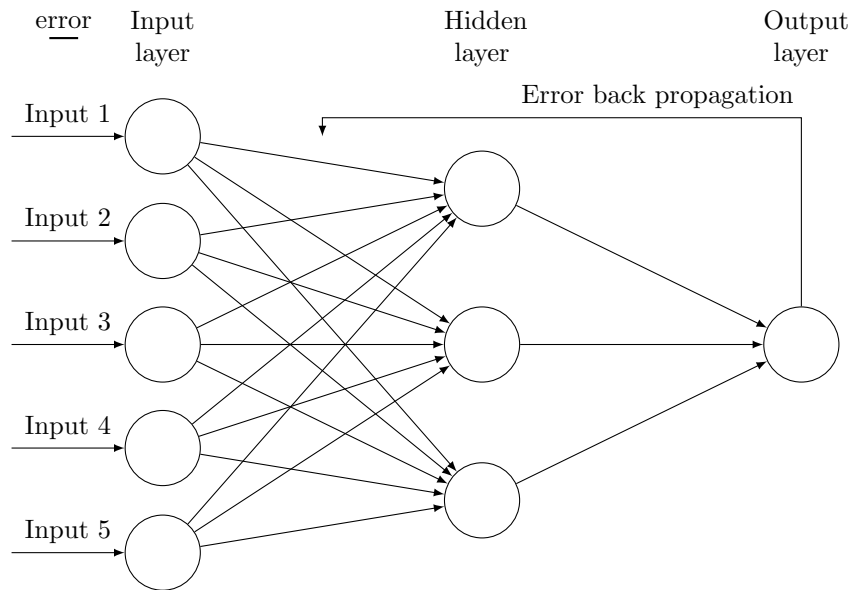
Example

Why do we compute absolute mean error over multiple points We have multiple data points. In order to comprehensively evaluate our model, we need to evaluate it over multiple data points. So mean absolute error is used to compute error over multiple data points. Another option is to compute root mean square error.

2.8 Back Propagation

Back Propagation is the process of adjusting weights based on the error. Remember that error is the difference between actual output and the estimated output from the NN.

So as we move backwards from output layer to hidden layers and then to the input layers, we adjust weights which will give them a close to expected output. The figure below explains the process.



Three algorithms for back propagation are used.

1. Gradient Descent
2. Stochastic Gradient Descent
3. Mini Batch Gradient Descent

The latter two algorithms are variants of the Gradient Descent Algorithm. We will first study the gradient descent algorithm.

2.8.1 Gradient Descent Algorithm

The gradient descent algorithm allows us to adjust weights in order to reduce error.

The algorithm works as follows:

1. Select initial weights
2. Compute slope of the error function
3. choose the steepest slope.
4. Choose the descent direction (adjust weights) opposite to the direction of the slope
5. the next value of weight is computed using the product of slope and the learning rate, where the learning rate is the magnitude of adjustment.
6. 1-5 iterate until convergence

The algorithm starts with an initial point and then repeatedly takes a step opposite to the gradient direction of the function at the current point. The algorithm can be written as follows $f(x)$ is as follows:

```

for  $k = 0, 1, 2, \dots$  do
   $g_k \leftarrow \nabla f(x_k)$ 
   $x_{k+1} \leftarrow x_k - t_k g_k$ 
end for

```

In summary, difference of weights (or weights adjustments) can be mentioned as follows:

$$\Delta W_i = -\alpha \frac{\partial E}{\partial W_i}$$

In the above equation, α gives us the amount of step size; whereas,

$$\frac{\partial E}{\partial W_i}$$

helps in determining the slope. The goal is to select the steepest slope in order to get the desired path to global minima. The slope helps in finding the direction towards the global minima.

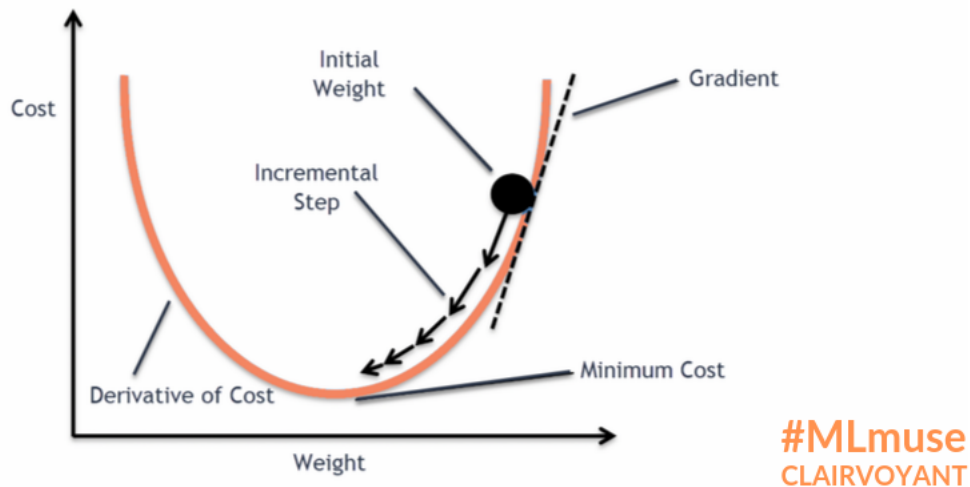


Figure 9: Gradient Descent

Figure 9 explains gradient descent algorithm. The error function (cost function) has been plotted as a function weights. Initial weights are randomly selected. Incremental steps are shown. The magnitude of these steps are determined by α , the learning rate, whereas the direction is selected opposite to the gradient. The algorithm will converge after determining the minimum cost.

In addition to local minima and the (rare) possibility of getting stuck at a saddle point, there are other issues that should be taken into consideration.

For example, if the step size t_k remains large, it may lead to an oscillatory behavior that does not converge.

Another issue is that, depending on the function and starting point, gradient descent could continue indefinitely because there is no minimum. Consider for example minimizing e^x : there is no finite x for which $\frac{d}{dx}e^x = 0$. Other functions could have such asymptotic minima but also a global minimum of lower value; gradient descent, depending on its starting point, might forever chase the asymptote, unaware of the true answer elsewhere in the search space.

Example

Why do we adjust weights in the opposite direction of slope Our aim is to minimize the value of loss function. Therefore, we would like to move in the direction where the loss function decreases the most. This is the opposite direction of slope.

2.8.2 Variants of Gradient Descent

An optimization algorithm determines how the weights of a neural network are updated. We study three commonly used variants of gradient descent.

1. **Batch Gradient Descent:** The gradient is computed using the **entire training dataset**, and the weights are updated only after one complete pass over all data points. For example, if the training dataset contains 1000 samples, the weights are updated after processing all 1000 samples.

Pros

- (a) Stable descent
- (b) Fewer oscillations

Cons

- (a) May get stuck in local minima
- (b) Extremely slow for large datasets

2. **Stochastic Gradient Descent (SGD):** A single data point is randomly selected, and the gradient is computed with respect to that point. SGD introduces randomness into the optimization process and is computationally efficient for large datasets.

Pros

- (a) Faster than Batch Gradient Descent
- (b) Can converge quickly
- (c) Has a higher chance of escaping local minima

Cons

- (a) High oscillations
- (b) May not converge exactly to the global minimum

3. **Mini-Batch Gradient Descent:** The training dataset is divided into smaller subsets called *mini-batches*. The model is trained on one mini-batch at a time. Each mini-batch may reach a local minimum, and the overall training process approximates a global minimum. This behavior is especially relevant for non-convex functions.

Figure 10 illustrates the difference between concave and non-concave functions.

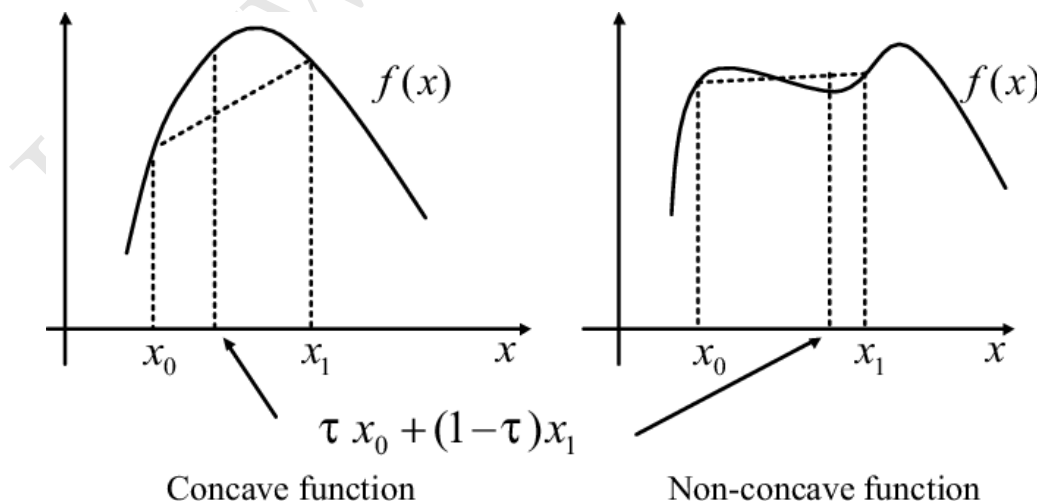


Figure 10: Concave vs Non-concave Function

Example**Differentiate between epochs, iterations, and batch size**

Epoch: One complete pass of the entire training dataset through the network, consisting of one forward pass and one backward pass.

Batch Size: The number of training samples processed in one forward/backward pass.

Iteration: One update of model parameters using a single batch.

Example: If the dataset contains 2000 samples and the batch size is 500, then one epoch consists of 4 iterations.

2.8.3 Chain Rule

Suppose that we have two functions $f(x)$ and $g(x)$ and they are both differentiable.

1. If we define $F(x) = (f \circ g)(x)$ then the derivative of $F(x)$ is,

$$F'(x) = f'(g(x)) \cdot g'(x)$$

2. If we have $y = f(u)$ and $u = g(x)$ then the derivative of y is,

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}$$

Figure 11: Chain rule explained

(a) $f(x) = \sin(3x^2 + x)$ [Hide Solution](#) ▼

It looks like the outside function is the sine and the inside function is $3x^2 + x$. The derivative is then.

$$f'(x) = \underbrace{\cos}_{\text{derivative of outside function}} \underbrace{(3x^2 + x)}_{\text{leave inside function alone}} \underbrace{(6x + 1)}_{\text{times derivative of inside function}}$$

Or with a little rewriting,

$$f'(x) = (6x + 1) \cos(3x^2 + x)$$

(b) $f(t) = (2t^3 + \cos(t))^{50}$ [Hide Solution](#) ▼

In this case the outside function is the exponent of 50 and the inside function is all the stuff on the inside of the parenthesis. The derivative is then.

$$\begin{aligned} f'(t) &= 50(2t^3 + \cos(t))^{49} (6t^2 - \sin(t)) \\ &= 50(6t^2 - \sin(t))(2t^3 + \cos(t))^{49} \end{aligned}$$

Figure 12: Chain rule examples

For the above figure, it is worth noting that derivative of $\cos x$ is $-\sin x$, whereas the derivative of $\sin x$ is $\cos x$.

2.8.4 Learning BackPropagation**2.8.5 Vanishing Gradient Problem**

The **vanishing gradient problem** occurs when gradients become very small as they are propagated backward through the network during training. As a result, early layers of a deep network learn very slowly or stop learning altogether.

This problem is especially common when using activation functions such as sigmoid or tanh, whose derivatives are less than one.

Why it happens During backpropagation, gradients are computed using the chain rule. If multiple derivatives with values less than one are multiplied together, the resulting gradient shrinks exponentially as we move toward earlier layers.

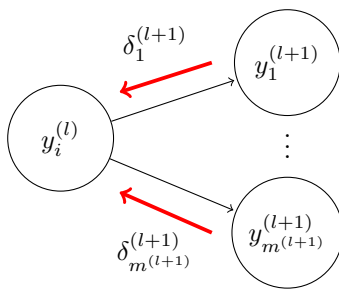


Figure 13: Once evaluated for all output units, the errors $\delta_i^{(L+1)}$ can be propagated backwards.

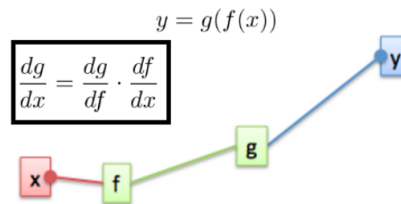


Figure 14: back propagation example

Effect on training

- Weights in early layers receive very small updates
- Network fails to learn meaningful features
- Training becomes extremely slow or stagnates

Mitigation strategies

- Use ReLU or Leaky ReLU activation functions
- Proper weight initialization
- Batch normalization

3 CNN for MLP

Multi-layer Perceptron provides us a reasonable accuracy for classification problems. But does it provide useful mechanism for image classification? Following challenges exists:

1. Images are 2-dimensional
2. Images vary in sizes
3. Images have high dimensions in pixels
4. Images have spatial information

We can transform a 2-D image into a 1-D image and input into an MLP. Further, we can apply padding to cater the size issue. This will solve 1) and 2).

However, using MLP for image classification will introduce scalability issues as we need a large number of neurons in the subsequent layers. For example if an image is of 128x128 size, then total no. of pixels are 16384. Similarly, spatial location will be lost during this conversion.

3.1 Convolution

Convolution can help us in feature extraction. Let us understand the process of convolution through the following figure.

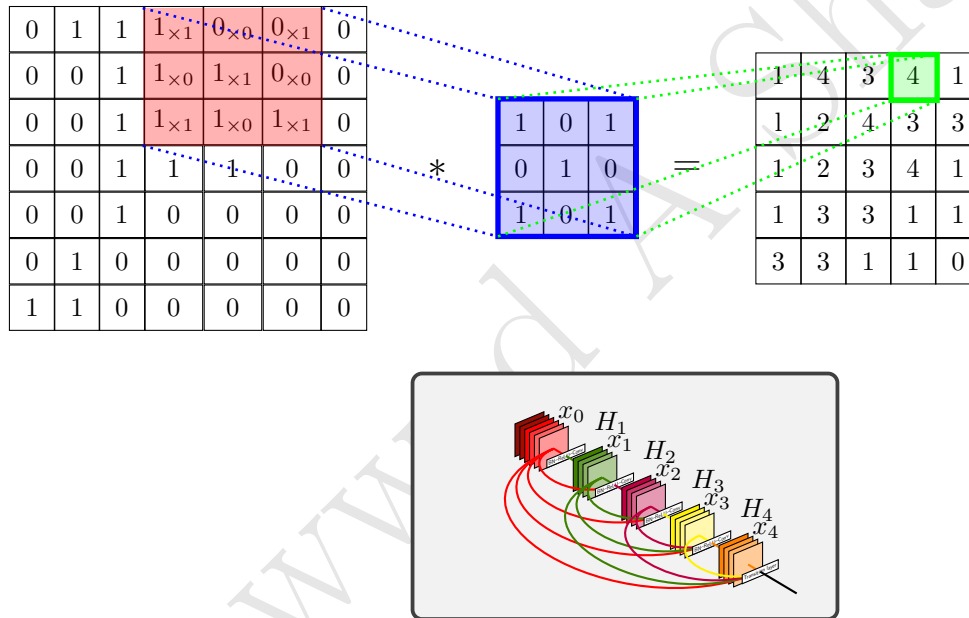
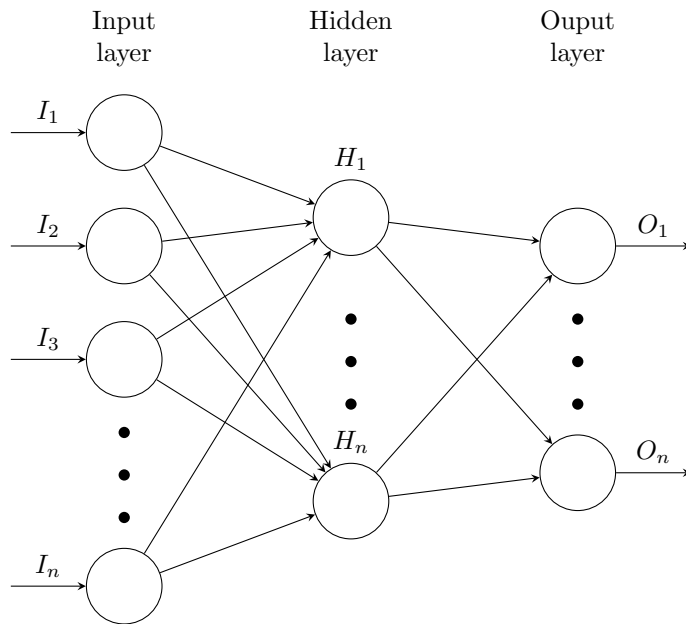


Figure 15: Dense connectivity pattern in a convolutional neural network

3.2 Classification after convolution

The process of classification can be followed after the convolution is complete. The figure below illustrates the process.



3.3 Convolution Process with an RGB Image

Let us understand the process of convolution. A digital image is typically represented as a three-dimensional array: **Height x Width x Channels**. For an RGB image, there are three channels:

- **Red (R) Channel**
- **Green (G) Channel**
- **Blue (B) Channel**

Each channel contains pixel intensity values. When a convolution operation is applied, a **filter (kernel)** slides over each channel separately, performing an element-wise multiplication followed by summation.

Example: Applying a 2x2 Filter to an RGB Image

Given:

- **Input Image:** A image (4x4 pixels, 3 channels)
- **Filter:** A kernel (2x2 spatial size, 3 channels)
- **Stride:** 1 (moves one pixel at a time)
- **Padding:** 0 (no extra padding around the image)

Steps:

1. The filter consists of three separate **2x2 matrices**, each applied to one of the RGB channels.
2. For each channel, the corresponding 2x2 matrix performs an element-wise multiplication and sums the results.
3. The summed values from the three channels are then added together to form a single output pixel value.
4. The filter slides across the image, computing values for the next positions, ultimately forming a feature map (output of the convolution operation).

Lets have a **3x3x3 filter** with distinct values and show how it applies to a **3-channel (RGB) image**.

Example: Applying a 3x3x3 Filter to an RGB Image

We'll take:

- A **3x3x3 filter** (with distinct values in each channel).

- A 3×3 RGB image (3 channels: Red, Green, Blue).

Step 1: Define the RGB Image ($3 \times 3 \times 3$)

Each color channel has its own 3×3 matrix:

Red Channel (R)

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Green Channel (G)

$$\begin{bmatrix} 10 & 11 & 12 \\ 13 & 14 & 15 \\ 16 & 17 & 18 \end{bmatrix}$$

Blue Channel (B)

$$\begin{bmatrix} 19 & 20 & 21 \\ 22 & 23 & 24 \\ 25 & 26 & 27 \end{bmatrix}$$

Step 2: Define the $3 \times 3 \times 3$ Filter

Each channel of the filter has a separate 3×3 matrix:

Red Channel Filter

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Green Channel Filter

$$\begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Blue Channel Filter

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Step 3: Perform Convolution

Convolution is done by:

1. **Element-wise multiplication** between each channel of the filter and the corresponding channel of the image.
2. **Summing up** the results from all three channels.

Red Channel Contribution

$$\begin{aligned} & (1*1 + 0*2 + (-1)*3) + \\ & (1*4 + 0*5 + (-1)*6) + \\ & (1*7 + 0*8 + (-1)*9) \\ & = (1 + 0 - 3) + (4 + 0 - 6) + (7 + 0 - 9) \\ & = -2 + (-2) + (-2) \\ & = -6 \end{aligned}$$

Green Channel Contribution

$$\begin{aligned} & (0*10 + 1*11 + 0*12) + \\ & (0*13 + 1*14 + 0*15) + \\ & (0*16 + 1*17 + 0*18) \\ & = (0 + 11 + 0) + (0 + 14 + 0) + (0 + 17 + 0) \\ & = 11 + 14 + 17 \\ & = 42 \end{aligned}$$

Blue Channel Contribution

$$\begin{aligned} & (-1*19 + 0*20 + 1*21) + \\ & (-1*22 + 0*23 + 1*24) + \\ & (-1*25 + 0*26 + 1*27) \\ & = (-19 + 0 + 21) + (-22 + 0 + 24) + (-25 + 0 + 27) \\ & = 2 + 2 + 2 \\ & = 6 \end{aligned}$$

Step 4: Compute Final Value

Now, sum the contributions from all channels:

$$\begin{aligned}\text{Final Output} &= (-6) + (42) + (6) \\ &= 42\end{aligned}$$

Thus, the output feature map at this position will have **42**.

In the above example, each filter has 10 parameters to learn. That is 3x3 and 1 parameter for bias.

3.4 Convolution and Activation in CNNs

In a Convolutional Neural Network (CNN), the convolution operation is followed by a non-linear activation function to produce a feature map.

Step 1: Convolution Given an input feature map X and a filter (kernel) W , the convolution operation computes:

$$Z = W * X + b$$

where:

- $*$ denotes the convolution operation,
- b is the bias term,
- Z is the linear output (pre-activation feature map).

Step 2: Activation Function The linear output Z is then passed through a non-linear activation function:

$$A = f(Z)$$

In most modern CNNs, the activation function used is the **Rectified Linear Unit (ReLU)**:

$$\text{ReLU}(z) = \max(0, z)$$

Thus, the final feature map becomes:

$$A = \text{ReLU}(W * X + b)$$

Why Activation is Necessary

- Introduces non-linearity into the network.
- Enables the model to learn complex patterns.
- Prevents the network from behaving like a single linear model.

Summary: Convolution extracts linear features, and ReLU introduces non-linearity to produce the final feature map used by the next layer.

Numerical Example: Convolution Followed by ReLU

Consider a 4×4 input feature map:

$$X = \begin{bmatrix} 1 & 2 & 0 & 3 \\ 4 & 1 & 2 & 1 \\ 0 & 1 & 3 & 2 \\ 2 & 3 & 1 & 0 \end{bmatrix}$$

Let the 2×2 filter (kernel) be:

$$W = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$$

Assume:

- Stride = 1
- No padding
- Bias $b = 0$

Step 1: Convolution The output size will be:

$$(4 - 2 + 1) \times (4 - 2 + 1) = 3 \times 3$$

Compute the first element:

$$\begin{aligned} Z_{11} &= \begin{bmatrix} 1 & 2 \\ 4 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} \\ &= (1)(1) + (2)(-1) + (4)(0) + (1)(1) \\ &= 1 - 2 + 0 + 1 = 0 \end{aligned}$$

Compute the second element:

$$\begin{aligned} Z_{12} &= \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} \\ &= (2)(1) + (0)(-1) + (1)(0) + (2)(1) \\ &= 2 + 0 + 0 + 2 = 4 \end{aligned}$$

Repeating this process for all positions gives:

$$Z = \begin{bmatrix} 0 & 4 & -1 \\ 3 & -1 & 3 \\ -1 & 3 & 2 \end{bmatrix}$$

Step 2: Apply ReLU Activation ReLU is defined as:

$$\text{ReLU}(z) = \max(0, z)$$

Applying ReLU to Z :

$$A = \begin{bmatrix} 0 & 4 & 0 \\ 3 & 0 & 3 \\ 0 & 3 & 2 \end{bmatrix}$$

Final Output The feature map after convolution and ReLU is:

$$A = \text{ReLU}(W * X + b)$$

Observation: Negative values are suppressed to zero, introducing non-linearity and allowing the network to learn complex patterns.

3.5 Padding

Padding refers to the process of adding extra pixels (typically zeros) around the border of an input feature map before applying convolution.

Why Padding is Needed Padding serves three important purposes:

1. Preserving Spatial Dimensions

Without padding, the spatial dimensions shrink after every convolution. For example, with input size n , filter size f , stride $S = 1$, and no padding ($P = 0$):

$$\text{Output size} = n - f + 1$$

Thus, spatial resolution decreases.

By choosing padding:

$$P = \frac{f - 1}{2}$$

(for odd-sized filters and $S = 1$), the output size becomes:

$$\text{Output size} = n$$

This is called “**same padding**”.

It ensures that the input and output have the same spatial dimensions.

2. Reducing Boundary Information Loss

Without padding, pixels near the boundary are used fewer times in convolution compared to central pixels.

Central pixels participate in many convolution operations, while corner pixels participate in only a few.

This makes boundary pixels less influential in feature extraction.

Padding ensures that:

- Every pixel (including boundary pixels) participates more equally in convolution.
- Edge information is preserved.

3. Variable Input size

Padding can also help to cater variable input size.

Types of Padding

- **Valid Padding:** $P = 0$ (no padding)
- **Same Padding:** Output size equals input size
- **Zero Padding:** Borders filled with zeros (most common)

Example Input: 5×5

Filter: 3×3

Stride: 1

Without padding:

$$\frac{5 - 3 + 0}{1} + 1 = 3$$

Output: 3×3

With padding $P = 1$:

$$\frac{5 - 3 + 2(1)}{1} + 1 = 5$$

Output: 5×5

Summary Padding helps maintain spatial resolution and ensures that boundary pixels are not underrepresented during convolution. It is a crucial design choice in deep CNN architectures.

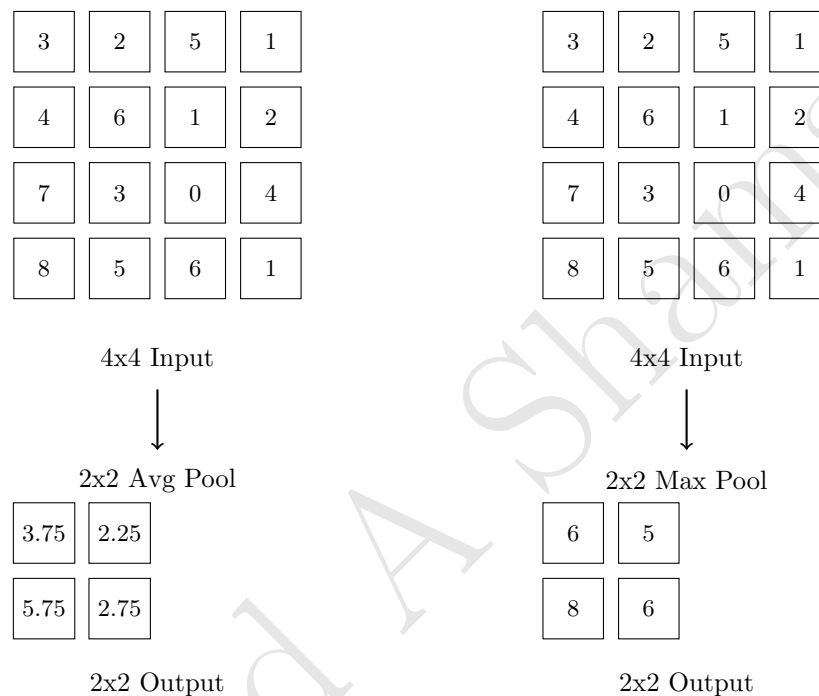
Average Pooling**Max Pooling**

Figure 16: Comparison of Average Pooling and Max Pooling (Kernel = 2x2, Stride = 2)

3.6 Pooling

IN CNN, the process of convolution is followed by 2-D pooling. The pooling can either be max pooling or average pooling. The figure below explains the difference..

Purpose of Pooling

Pooling is used in Convolutional Neural Networks (CNNs) to reduce the spatial dimensions (height and width) of feature maps while preserving the most important information.

- Reduces computational cost in deeper layers.
- Decreases the number of parameters.
- Helps control overfitting.
- Provides local translation invariance (robustness to small shifts).

In short, pooling performs **feature reduction** while retaining the most salient features of the input.

Multiple Rounds of Convolution and Pooling in CNNs

In Convolutional Neural Networks (CNNs), convolution and pooling operations are typically applied in multiple stages (or blocks). A common pattern is:

Convolution $\rightarrow$ Activation (ReLU) $\rightarrow$ Pooling

This structure is repeated several times before the final fully connected layers.

Why Multiple Rounds? Each stage extracts increasingly abstract features:

- **Early layers:** Detect low-level features such as edges and textures.
- **Intermediate layers:** Capture patterns and object parts.
- **Deeper layers:** Learn high-level semantic features (e.g., faces, objects).

As the network goes deeper:

- Spatial dimensions (height and width) decrease.
- Number of feature maps (channels) typically increases.
- Features become more abstract and task-specific.

Typical CNN Structure

Conv $\rightarrow$ ReLU $\rightarrow$ Pool
 Conv $\rightarrow$ ReLU $\rightarrow$ Pool
 Conv $\rightarrow$ ReLU
 Fully Connected $\rightarrow$ Softmax

Important Note Pooling is common but not mandatory after every convolution layer. Some modern architectures reduce pooling usage or replace it with strided convolutions. In some cases, *global average pooling* is used before the final classification layer.

Summary: CNNs progressively reduce spatial resolution while increasing feature abstraction through repeated convolution and pooling blocks.

3.7 Understanding how Features are calculated at each layer

Figure 17: CNN Features Figure 17 illustrates the hierarchical feature extraction process in a Convolutional Neural Network (CNN) through successive convolution, pooling, and dense layers.

The input image of size $150 \times 150 \times 3$ is first processed through convolutional layers that learn spatial features using multiple filters. Each convolutional layer increases the number of feature maps (channels), enabling the network to capture increasingly complex patterns.

Max pooling layers reduce the spatial dimensions of the feature maps (e.g., $148 \times 148 \rightarrow 74 \times 74 \rightarrow 36 \times 36$), while preserving the most salient information. This progressive reduction lowers computational complexity and introduces translation invariance.

As the network goes deeper:

- Spatial resolution decreases.
- Number of feature maps increases (e.g., $16 \rightarrow 32 \rightarrow 64$).
- Features become more abstract and task-specific.

After the final convolution and pooling stages, the feature maps are flattened into a one-dimensional vector and passed to fully connected (dense) layers for classification.

Thus, Figure 17 demonstrates how a CNN transforms raw pixel data into high-level learned representations suitable for decision making.

Model: "model_1"

Layer (type)	Output Shape	Param #
input_2 (InputLayer)	[(None, 150, 150, 3)]	0
conv2d_3 (Conv2D)	(None, 148, 148, 16)	448
max_pooling2d_3 (MaxPooling 2D)	(None, 74, 74, 16)	0
conv2d_4 (Conv2D)	(None, 72, 72, 32)	4640
max_pooling2d_4 (MaxPooling 2D)	(None, 36, 36, 32)	0
conv2d_5 (Conv2D)	(None, 34, 34, 64)	18496
max_pooling2d_5 (MaxPooling 2D)	(None, 17, 17, 64)	0
flatten_1 (Flatten)	(None, 18496)	0
dense_2 (Dense)	(None, 512)	9470464

Figure 17: CNN Features

Computation of Parameters in Convolutional Layers

In a convolutional layer, the number of learnable parameters depends on:

- Filter size ($f \times f$)
- Number of input channels (C_{in})
- Number of filters / output channels (C_{out})

General Formula:

$$\text{Parameters} = (f \times f \times C_{in}) \times C_{out} + C_{out}$$

The additional C_{out} term corresponds to one bias parameter per filter.

Layer 1: Conv2D (16 Filters)

Input size: $150 \times 150 \times 3$

Filter size: 3×3

Number of filters: 16

$$(3 \times 3 \times 3) \times 16 + 16$$

$$= 27 \times 16 + 16$$

$$= 432 + 16$$

$$= 448$$

Total Parameters = 448

Layer 2: Conv2D (32 Filters)

Input channels now equal 16 (from previous layer).

Filter size: 3×3

Number of filters: 32

$$(3 \times 3 \times 16) \times 32 + 32$$

$$= 144 \times 32 + 32$$

$$= 4608 + 32$$

$$= 4640$$

Total Parameters = 4640

Layer 3: Conv2D (64 Filters)

Input channels now equal 32.

Filter size: 3×3

Number of filters: 64

$$(3 \times 3 \times 32) \times 64 + 64$$

$$= 288 \times 64 + 64$$

$$= 18432 + 64$$

$$= 18496$$

Total Parameters = 18496

Parameters in Dense Layer

After the final pooling layer, suppose the feature maps are flattened into a vector of size 18496 and connected to a dense layer of 512 neurons.

$$\text{Parameters} = (18496 \times 512) + 512$$

$$= 9,470,464$$

More than 99% of parameters are in the dense layer

Observation:

- Convolutional layer parameters depend on kernel size and channel depth.
- They do NOT depend on spatial dimensions (height and width).
- Dense layers typically dominate the total parameter count because each neuron connects to all neurons in the previous layer.

3.8 How the size of feature map is calculated - Dimensionality Reduction**Dimensionality Reduction in Convolution**

In a convolutional layer, the spatial dimensions of the input feature map change depending on the filter size, padding, and stride.

Output Dimension Formula If the input has spatial dimension $n \times n$, and we apply:

- Filter size: $f \times f$
- Padding: P
- Stride: S

then the output spatial dimension is given by:

$$\text{Output Size} = \left\lfloor \frac{n - f + 2P}{S} \right\rfloor + 1$$

This formula is applied separately to both height and width.

Explanation of Terms

- n : Input size (height or width)
- f : Filter (kernel) size
- P : Padding applied to the input
- S : Stride (step size of filter movement)

Example 1: No Padding Input: 150×150

Filter: 3×3

Stride: 1

Padding: 0

$$\frac{150 - 3 + 0}{1} + 1 = 148$$

Output: 148×148

Example 2: With Padding Input: 150×150

Filter: 3×3

Stride: 1

Padding: 1

$$\frac{150 - 3 + 2(1)}{1} + 1 = 150$$

Output: 150×150

Thus, padding can preserve spatial dimensions.

Dimensionality Reduction Insight Dimensionality reduction occurs when:

- $S > 1$ (larger stride), or
- $P = 0$ with $f > 1$, or
- Pooling layers are applied.

As we go deeper in CNNs:

- Spatial dimensions decrease.
- Number of feature maps typically increases.

This allows the network to progressively reduce spatial resolution while increasing feature abstraction.

3.9 Normalization

It refers to changing (or scaling) the numerical values of the data set to put them along the same scale.

This is needed if the values in the input data are along different scales. For instance, age and salary. If we do not normalize data then salary is likely to be dominated.

standardization (or normalization) = $z = (x - m) / S$ for each data point x , we subtract it from mean and divide it by standard deviation. here m refers to the mean value whereas S refers to the standard deviation

The standardized data now has a mean of 0 and standard deviation of 1. Why do we do this? Some data points may be very high other may be very low.

Large variation in inputs may cascade to initial layers and cause imbalance gradient and exploding gradient problem. Normalized data means that data is trained at high speed as well and will also cater the above problem.

3.9.1

sectionScaling

Scaling is another type of normalization in which we scale the data. For instance, let's assume that we are working on age and salaries and there is huge variation. Through scaling, we can scale the data between 0 and 1. One way of doing scaling is to divide each data item by the maximum value in that column.

3.10 Vanishing/exploding gradient problem

Vanishing gradient is another challenging problem in training deep neural networks. In theory, as you go deeper in the network (increasing the layers) should reduce the training error. However, it is often not true in practice, due to vanishing or exploding gradients. Consider a situation, where the initial weights are either less than zero or greater than one, and we use a linear activation function with zero bias. As we go deeper in the network, slopes or derivatives can either be very big or very small.

Recall that the cost $J(w)$ function (the difference between the estimated output and the actual output) is reduced through back propagation. In that each parameter (W) is adjusted in proportion to its gradient with respect to cost. For instance, if the gradient is large and +ve then it implies that the parameter has high correlation with the cost and decreasing it will decrease the cost.

As we move from the final hidden layer to the first hidden layer, gradient is flattened and it vanishes. The farther the layer is from the output, the more slowly it tends to learn. If we keep adding more hidden layers, the farther layers will eventually stop learning. This is specifically true for sigmoid (and similar) activation functions which have an output space between 0 and 1. Figure 19 shows sigmoid and its derivative. When the input to the sigmoid activation function is large (or small), its output is close to one (or zero), and the derivative is close to zero. For neural networks with only a few hidden layers, this is acceptable. However, for large neural networks, the gradient could be too small. If we recall, that through the chain rule, the gradient is multiplied. For initial layers, this could be a bigger problem. When n hidden layers use an activation, n small derivatives are multiplied together. With small gradient, the value becomes too low for initial layers. This implies that the weights and biases of the initial layers will not be updated for effective range during the training phase. This can lead to overall inefficiency.

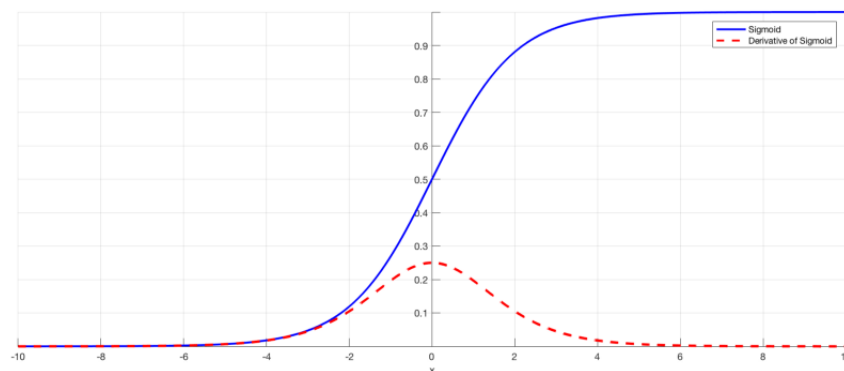


Figure 18: Sigmoid and Its derivative

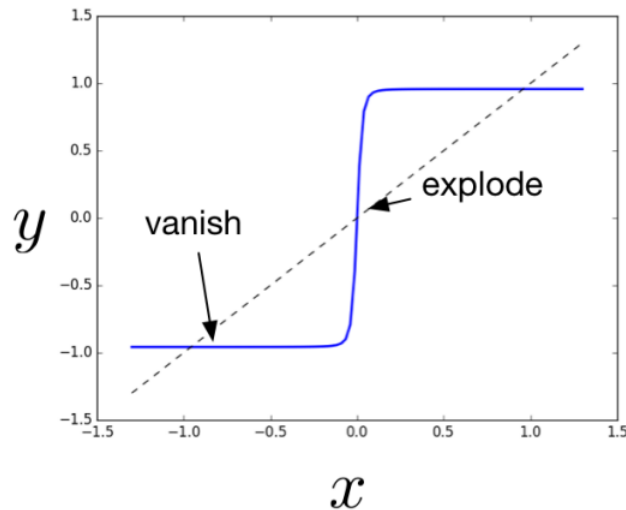


Figure 19: Vanishing and Exploding Gradient

There are a few tricks to deal with the vanishing or exploding gradient problem partially. One of them is the careful choice of weight initialization. Such as the variance of weights initialization may be equal to $1/n$, where n is the total number of weights in the layer. Another way is to use skip connections as in "ResNet" architectures. People often confuse the vanishing gradients problem with over-fitting, which is not correct. In over-fitting problems, the training error does not increase by increasing the number of layers, however in vanishing gradients, the training error also increases, as we go deeper in the network.

3.11 Batch Normalization

Often refers as BN as well. A technique to cater vanishing gradient problem.

Internal Covariate shift The distribution of hidden layers may gradually move around. Distribution of the inputs to layers somewhere down in the network may change after each mini-batch when the weights are refreshed.

weights are updated over each epoch. what if one of the weights becomes drastically larger over other weights. this will cause o/p from the corresponding neuron to be large and will cause instability and will cascade through the neural network

For BN, we use mean and SD over the batch instead of the complete data set.

BN is applied to layers which we chose to apply to. The purpose is to normalize the o/p from activation fn. Following are the steps:

1. mean of batch and SD $z=(x-m)/s$.
2. multiply the o/p to an arbitrary parameter g . $z*g$
3. add arbitrary parameter b to the resulting product $(z*g) + b$. sets new sd and mean for the data.

g is analogous to sd and b is analogous to mean four parameters m, s, g , and b will change and will ensure that data is normalized.

BN is focused on taking an activation output from the preceding layer, subtracting the batch mean, and dividing by the batch standard deviation. Thus if there are extreme values in the preceding layer, they will not cause exploding or vanishing gradient problem in the next layer.

Difference between normalization and batch normalization

1. The former is applied on the whole data, while the latter is applied on the batch data.
2. Normalization is for the data at the input. BN is within the network.

3.12 Overfitting

An overfit model has negligible error during the training phase, but has comparatively higher error during the testing phase. **One reason may be that the learning algorithms not only learn the patterns in the data but also the noise in the data. This problem is usually seen in deeper supervised learning algorithms.**

An underfit model has high error rate during the training phase. Figure ??fig:overfit explains the difference. Overfitting occurs when we try to fit all possible data points.

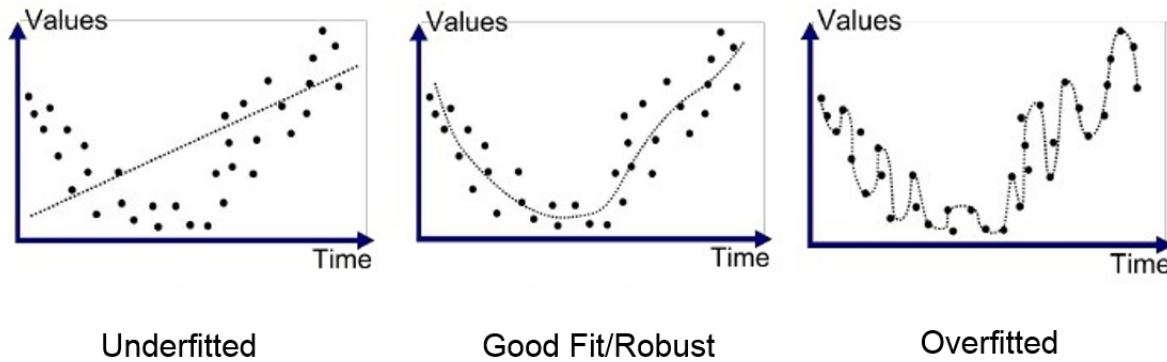


Figure 20: Overfitting vs. Underfitting

Following are the techniques to overcome the overfitting problem.

3.12.1 L1 and L2 Regularization

Penalize models for including parameters by including these parameters in the cost function. The larger the size of the parameter, the more will be the cost.

Consequently, parameters are not retained unless they contribute to the difference between estimated and actual values of y .

Difference between L1 and L2 is that L1 adds actual value of the parameter, whereas L2 adds the square value.

3.12.2 DropOut

It is another technique to solve the overfitting problem. Dropout prevents any neuron from becoming excessively influential. As compared to BN, however, dropout does not constrain parameter.

It assumes that a proportion of neurons in each layer does not exist. A round of iterations are followed. In each iteration, we randomly drop a certain percentage of neurons. This dropping is totally random, i.e., for a given layer, each iteration does not have any linkage with the dropout of previous iteration.

Once the training is completed, an additional step is needed in the validation process. Since during training, only a subset of neurons have been used, a proportion weight age is used as a multiplication factor during validation. That is, if for a given layer, 50% neurons have been dropped out during training, then during validation a multiplication factor of 0.5 will be used for the activation function of the neurons at that layer.

3.12.3 Data Augmentation

A third technique, which is used to cater the overfitting problem is data augmentation. It can also be used to generate more data for improved precision. It is specifically useful for Computer Vision techniques.

It will be useful for training.

Methods

1. Mirroring on the vertical axis. That is flipped it horizontally
2. Random Cropping. Crop an image. Not a perfect technique. Because randomly bad crop may be used. But reasonable

3. Rotation: rotate an image at different angles
4. Shearing along x-axis or y-axis
5. Color Shifting: Add to RGB channels
6. PCA color augmentation
7. Noisy versions of the images
8. Scaling

hyper parameters: parameters for random cropping

3.13 LeNet

Purpose: Hand Digit classification and recognition Commercial Use: read millions of cheques in banks Developed in 1998. Input: MNIST database size 32x32 distortions were introduced

3.14 Architecture

* Figure 21 illustrates the architecture of Lenet, while figure 22 illustrates layered-wise approach. The input image is a 32x32 gray scale.

The LeNet architecture has two blocks, a convolutional block and a dense-layer block. The convolutional block has two convolntional layers, each of them consists of the following:

1. A convolutional layer with multiple filters of size 5x5
2. Sigmoid activation function, and
3. A pooling operation

Note that for CNN, RELU activation function works better. **One reason for this is all negative values are replaced by zeros after applying the activation function.** However, at that time (1998), this was unknown. LeNet uses average pooling function.

3.14.1 convolutional block

* The first convolution layer used 6 filters with size of 5*5 with a stride of one and no padding. After that average pooling is applied with a size of 2*2. The second convolutional layer has 16 filters, each of size 5*5 with a stride of one and no padding. Again, average pooling is applied with a size 2*2.

3.14.2 Dense block

* In LeNet, the output of the second convolutional layer is flattened (after applying activation and pooling). The output is connected to a fully connected layer of 120 neurons, which is further connected to another layer of 84 neurons. The final layer has 10-classes, one for each digit. Softmax function is used as an activation function.

$$\sigma(z) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}} \quad (34)$$

As we go deeper, size of feature map shrinks (Hint: pooling) but the no. of feature maps increase.

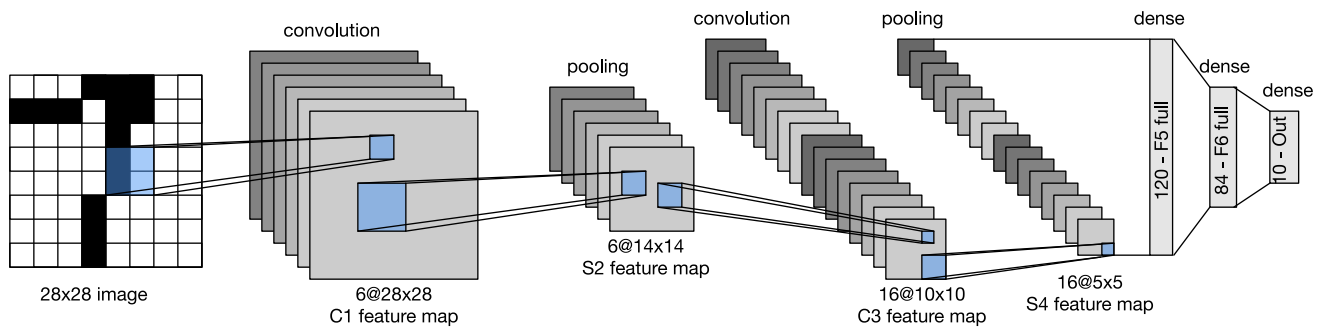


Figure 21: Lenet Architecture proposed by Yann Lecunn in 1998 is the pioneer convolutional neural network for image recognition. It was trained on gray scale images and on MNIST handwritten digit dataset.

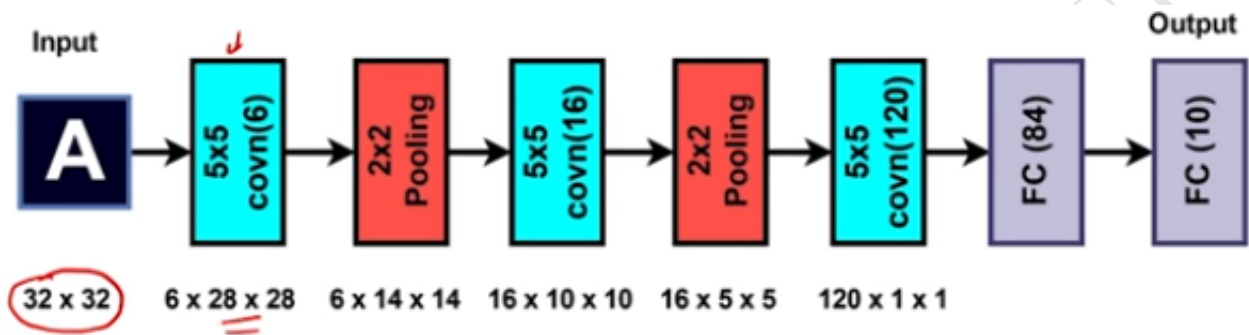


Figure 22: Lenet Architecture - layers

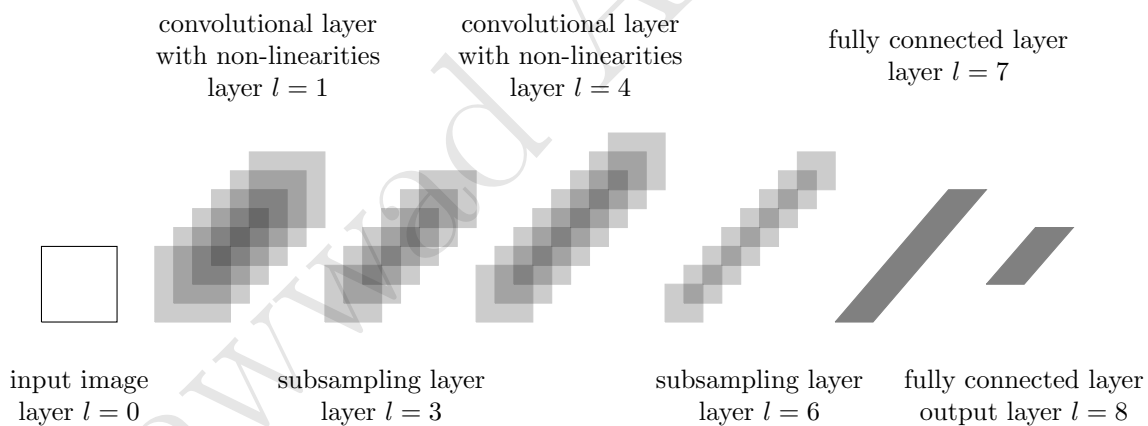


Figure 23: The architecture of the original convolutional neural network, as introduced by LeCun et al. (1989), alternates between convolutional layers including hyperbolic tangent non-linearities and subsampling layers. In this illustration, the convolutional layers already include non-linearities and, thus, a convolutional layer actually represents two layers. The feature maps of the final subsampling layer are then fed into the actual classifier consisting of an arbitrary number of fully connected layers. The output layer usually uses softmax activation functions.

4 Inception Network

In the field of CNN classifiers, the Inception network was a major breakthrough moment. Most common CNNs, prior to inception, simply stacked convolution layers deeper and deeper in the hopes of improving efficiency. However, in real time applications, the size of important sections of the picture will vary dramatically. For example, a dog image may be one of the following, as shown below. In each picture, the dog occupies a different location.

Choosing the right kernel /filter size for the convolution operation becomes difficult due to these large variation

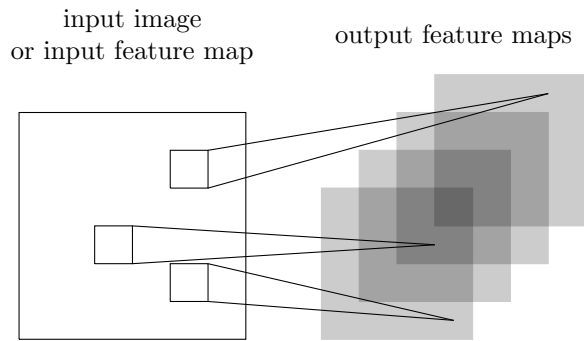


Figure 24: Illustration of a single convolutional layer. If layer l is a convolutional layer, the input image (if $l = 1$) or a feature map of the previous layer is convolved by different filters to yield the output feature maps of layer l .

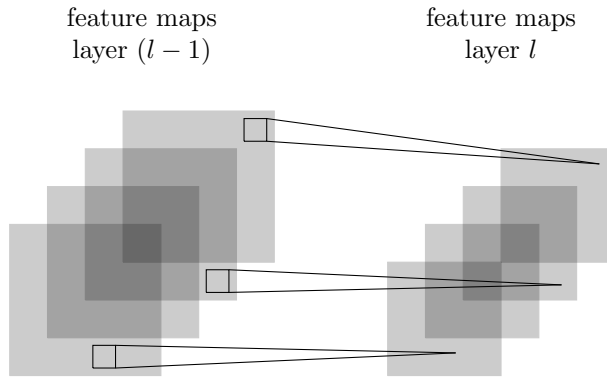


Figure 25: Illustration of a pooling and subsampling layer. If layer l is a pooling and subsampling layer and given $m_1^{(l-1)} = 4$ feature maps of the previous layer, all feature maps are pooled and subsampled individually. Each unit in one of the $m_1^{(l)} = 4$ output feature maps represents the average or the maximum within a fixed window of the corresponding feature map in layer $(l - 1)$.

in the position of the salient objects. For information, that is distributed more globally (uniformly over the entire image), a larger kernel is preferred, whereas for information that is distributed more locally, a smaller kernel is preferred.

Overfitting is an issue for very deep networks. It's also difficult to propagate gradient updates through the entire network. Stacking massive convolution operations in an inefficient manner is computationally costly.

4.1 Filter Size

We can choose different size of filters. For instance, 5×5 , 3×3 , and 1×1 . These three sizes can be used to detect different sizes of objects.

5×5 can detect larger objects, 3×3 can detect medium-sized objects, whereas 1×1 can detect small objects. Figure 27 explains the concept.

The three different size of filters should produce the same size of feature map. This is because the convolution from the three sizes will be cascaded together. In other words X and Y dimensions of the three feature maps produced through convolution of these three filters (5×5 , 3×3 , and 1×1) should be same. The Z dimension will be cascaded together.



From left: A dog occupying most of the image, a dog occupying a part of it, and a dog occupying very little space (Images obtained from [Unsplash](#)).

Figure 26: Predicted vs actual output

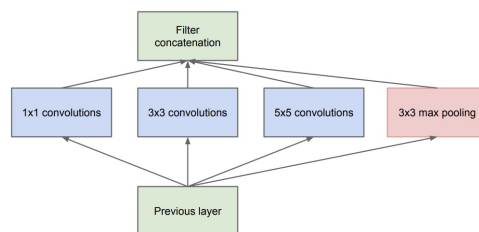


Figure 27: Inception

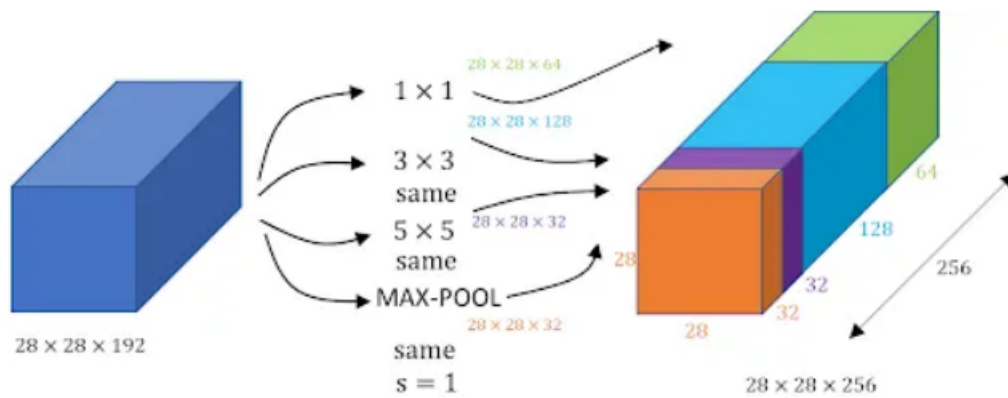


Figure 28: Inception Example

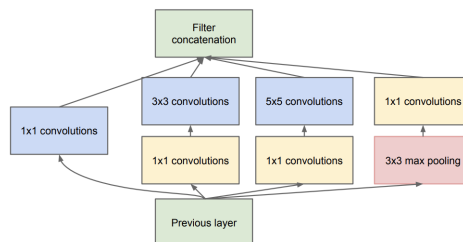


Figure 29: Adding 1x1 Conv

Example**Similar size of feature maps**

To yield similar size feature maps, we should use padding.

Remember the size of feature map is given by

$$\text{sizeof featuremap} = (n + 2(P) - f) / S + 1$$

where n=size of input f=size of filter p=size of padding s= stride

A 5X5X192 filter on the image size of 28X28X192 with no padding and single stride will give us $28+2(0)-5/1 + 1$ will give us the output of 24 x 24

Similarly, 3X3 filter will yield a feature map of size 26X26 and a 1x1 filter will yield a feature map of size 28X28.

In order to have a similar size, we can apply 2X2 padding on the input for 5X5 filter and 1X1 padding on images with 3X3 filter.

Figure 30 illustrates the concept of adding of 1X1 conv before the actual filter of 5x5, 3x3, and 1x1.

Example**How 1X1 conv helps**

A 5X5 filter with 2X2 padding on a 28X28 input image will yield a 28x28 feature map. Total number of operations are $5 \times 5 \times 192 \times 28 \times 28 \times 32 = 120$ Million.

Remember that 5X5X192 is the dimension of the input filter, whereas 28X28X32 is the resultant feature map. In that 32 is the no of 5x5 filters being convoluted.

This number can be reduced by applying a 1x1 conv first

If we apply 16 filters of 1x1x192, then the resultant feature map will be 28X28X16. Applying 32 filters of 5X5X16 on the this feature map (after padding), will result in the feature map of 28X28X32. However the total no of operations will be

1) $28 \times 28 \times 16 \times 1 \times 1 \times 192 = 2.4$ Million 2) $28 \times 28 \times 32 \times 5 \times 5 \times 16 = 10$ Million

Total = 12.4 million. This is a 10-time reduction.

Similarly inception with 1X1 conv. can be applied before 3X3 and 1X1.

The whole idea can be understood from figure 30

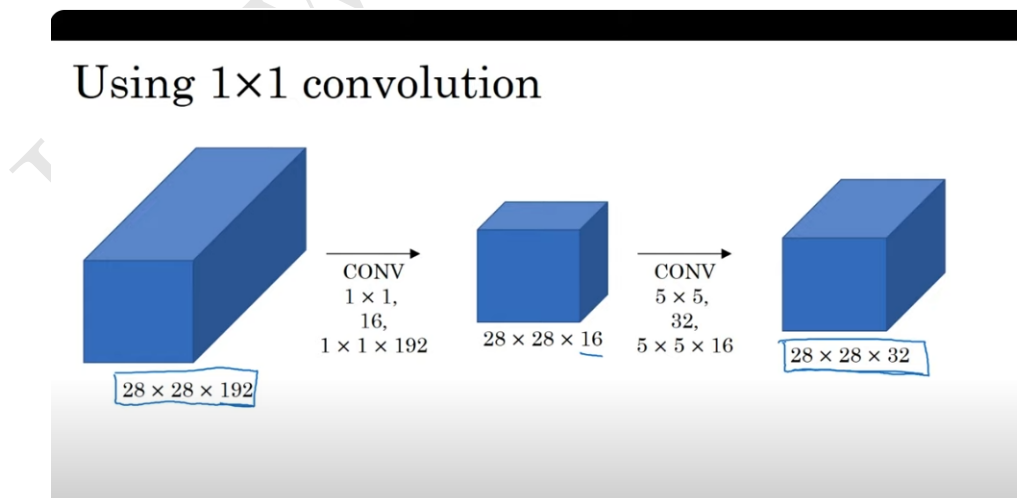


Figure 30: 1x1 and 5x5